



PulseCoach: Software Design & Test Campaign

DESIGN & TEST DOCUMENT

DESIGN AND IMPLEMENTATION OF MOBILE APPLICATIONS

Paolo Limonta, Daniel Moraru

Advisor:

Prof. Luciano Baresi

Co-advisors:

Davide Yi Xian Hu

Academic year:

2025-2026

Abstract: PulseCoach is a Flutter/Dart cross-platform application (Android, iOS/iPadOS, Wear OS) that generates personalized micro-workout sessions of 2–10 minutes, adapted in real time to the user’s physiological state, behavioural history, and environmental context. Its distinguishing property is that the adaptive intelligence runs *entirely on-device*: a deterministic safety layer, a behavioural state machine, and a lightweight ϵ -greedy multi-armed bandit over a safety-constrained action set cooperate to produce a small, safe, personalized daily plan together with a structured, human-readable explanation of every recommendation. This document is the Software Design Description and Test Campaign report for the system. It presents the architecture (Clean Architecture with a framework-free AI engine), the design of the decision engine, the data and concurrency models, the resilience strategy for external integrations, the optional privacy-preserving cloud layer, and the multi-device presentation design. It then describes the verification strategy, a layered and risk-prioritized automated campaign, and its measured outcome: 1,420 passing automated tests across 183 files, with all eighteen subsystems clearing the quality gate. The document explains how the software is built and how it is verified; it is not a user manual and contains no screenshots.

Key-words: adaptive fitness, on-device AI, multi-armed bandit, Clean Architecture, Flutter, test campaign

1. Introduction

This document is the **Software Design Description (SDD)** and **Test Campaign report** for PulseCoach. Its two goals are to explain how the software is designed — its architecture, its module decomposition, the on-device decision engine, and the data model — and to present the verification strategy and its results: the layered test architecture, the risk model that drives it, and the traceability from requirements through to the campaign’s measured outcome. The structure follows the design-viewpoint organization recommended by IEEE Std 1016 [4] and the level/priority/coverage vocabulary of established software-testing practice [5]. It is written for a technical reader. It is not a user manual and, by requirement, contains no screenshots: the architecture is communicated through models and diagrams.

1.1. Product in one paragraph

PulseCoach generates personalized micro-workout sessions (2–10 minutes) adapted in real time to the user’s current physiology, recent training history, and surrounding conditions. It targets the moment before the decision to exercise: the user opens the app and receives a calibrated three-session daily plan in under 30 seconds, with a one-tap start. The differentiating feature is **explainability**: every recommendation surfaces a structured, human-readable reason (e.g. “intensity kept light: high perceived effort in your last two sessions”), projected from the same decision signals the engine actually consumes. The intelligence runs **entirely on-device**; no

biometric-derived data leaves the device in readable form (the optional cloud-phase backup uploads only client-side end-to-end-encrypted ciphertext; see Section 10), and no network connection is required to learn.

1.2. Design drivers

The architecture is shaped by four forces that recur throughout this document:

1. **On-device adaptive intelligence.** An ε -greedy multi-armed bandit, a behavioural state machine, and a deterministic safety layer must run locally, reproducibly, and off the UI thread.
2. **Privacy by construction.** Biometric-derived state never crosses the device boundary in cleartext; location is reduced to city level; the cloud phase is optional and additive and never a prerequisite for the free core.
3. **Offline-first resilience.** The local database is the source of truth; every external dependency (sensors, weather, exercise catalog, cloud) degrades gracefully and silently.
4. **Testability.** A large automated campaign ($>1,400$ tests) is only sustainable if every boundary is mockable and every time- or randomness-dependent computation is deterministic under test.

1.3. Scope of the built system

PulseCoach is a mature, feature-complete system. Table 1 summarizes the surface covered by the design.

Table 1: Delivered scope at a glance (application version 1.1.0).

Dimension	Realization	Metric
Requirements	Functional (core phase + cloud phase + polish) Non-functional	82 FR 39 NFR
Codebase	Production Dart under <code>lib/</code> (excl. generated)	$\approx 25,000$ LOC
Source files	Dart files under <code>lib/</code>	419
Feature modules	Clean-Architecture feature slices	9 (+3 cloud phase)
Platforms	Android, iOS/iPadOS, Wear OS companion	3 targets
External services	Open-Meteo, ExerciseDB, Supabase, Store IAP	4
Test suite	Automated test files / executed cases	183 / 1,421
Static analysis	<code>flutter analyze lib test</code>	0 issues

1.4. Definitions and abbreviations

Term	Meaning
RPE	Rate of Perceived Exertion (Borg-style subjective effort, 1–10) [2]. Post-session feedback signal.
Bandit	ε -greedy multi-armed bandit, the online learner that scores session “arms”; here the user context restricts the <i>feasible</i> arms rather than conditioning the value estimate [10].
Arm	A candidate session choice: <i>session type</i> $\times$ <i>intensity</i> (9 total).
EMA	Exponential Moving Average; the per-arm value update of the bandit (Eq. 2), learning rate α .
BLoC / Cubit	Business-Logic-Component pattern (<code>flutter_bloc</code>) for reactive state management.
Isolate	A Dart concurrency unit with an isolated heap; used to run the AI off the UI thread.
Drift	Type-safe, reactive local SQLite persistence layer for Dart/Flutter.
RLS	Row-Level Security (Postgres), the server-side privacy boundary in the cloud phase.
E2E	End-to-end encryption; the backup is encrypted client-side, so the server stores only opaque ciphertext.
OIDC	OpenID Connect; the federated sign-in protocol behind Google authentication in the cloud phase.
IAP	In-App Purchase; the platform billing channel for the Pro subscription (via RevenueCat).
pgTAP	A unit-testing framework for PostgreSQL; used to verify RLS policies and SQL scoring functions.
FR / NFR	Functional / Non-functional requirement.

2. Design goals, constraints and quality attributes

2.1. Functional overview

The functional surface spans ten categories (Table 2). The intellectually dense part, and the part treated in most depth here, is the adaptive planning core; the remainder is standard mobile CRUD, charting, settings, and, in the cloud phase, an optional cloud/social layer.

Table 2: Functional categories and their architectural weight.

Category	Requirements	Primary architectural impact
AI-powered daily planning	FR5–FR12	Bandit + state machine + safety rules; isolate execution; deterministic override layer
Explainability & trust	FR13–FR15	Structured reason projection from AI decision context
Guided session execution	FR16–FR20	Full-screen timer flow, haptics, notifications, Wear OS sync
Feedback & adaptation loop	FR21–FR25	RPE → reward → bandit update → state transition (closed loop)
Exercise catalog	FR26–FR28	ExerciseDB integration, cache, bundled fallback
Environmental context	FR33–FR36	Open-Meteo integration, TTL cache, city-level location
Multi-device experience	FR37–FR41	Responsive dual layout; Wear OS companion (display-only)
Sensor integration	FR42–FR44	Health API, accelerometer, GPS (all optional, RPE-only fallback)
Offline & data	FR45–FR47	Drift as source of truth; TTL caching; deferred sync queue
Onboarding, profile, settings, progress	FR1–FR4, FR29–FR32, FR48–FR52	Standard CRUD, charts, theming, export
<i>cloud phase: accounts, subscription, social</i>	FR53–FR77	Optional cloud: auth, IAP gating, social graph, real-time shared sessions
<i>core phase: experience polish</i>	FR78–FR82	In-session milestones, session notification, active-days indicator, factor icons

2.2. Non-functional requirements as architectural drivers

Several NFRs directly *cause* a design decision. Table 3 maps the load-bearing NFRs to the mechanism that satisfies them.

Table 3: Key non-functional requirements and their realizing mechanism.

NFR	Requirement	Realizing mechanism
NFR1	Plan generated in <30 s	On-device computation, no network on the critical path
NFR2	60 fps during AI computation	AI runs on a separate isolate via <code>compute()</code> ; UI thread never blocks
NFR4	Haptic latency <200 ms	Direct platform-channel haptics on step transitions
NFR6	Cold start <3 s	Lazy DI, deferred heavy work, shimmer-first rendering
NFR7,12	No biometric transmission; GDPR Art. 9	On-device-only processing of health-derived signals [3]
NFR8	City-level location only	Coordinates rounded before any weather API call
NFR11	No account required (free core)	Auth gates only cloud extras; the core phase is account-free
NFR13–NFR18	Offline-first, persistence, TTL	Drift primary store; cache tables with <code>cachedAt</code> ; deferred sync queue
NFR19–NFR23	Integration resilience	Fail-fast + cache-first + bundled default at every external boundary
NFR27,28	Auth security; on-device personalization	OAuth/OIDC + secure token storage; client-side E2E backup
NFR29,30	Visibility tiers; erasure cascade	Postgres Row-Level Security; <code>ON DELETE CASCADE</code> + Edge Function
NFR31	≈1 s real-time sync, drop-out tolerant	Supabase Realtime Broadcast + Presence, host authority
NFR34	Offline-first preserved under the cloud phase	Cloud is additive; core never blocked by cloud/account state
NFR36	EU data residency	Supabase EU region (Frankfurt)

2.3. Fixed technical constraints

The following constraints are exogenous (course rules, platform limits, product decisions) and bound the design space:

- Single Flutter/Dart codebase for all platform targets; `go_router` for navigation; `flutter_bloc` for state; `drift` for persistence.
- The core phase has *no* cloud backend: all intelligence is on-device. The cloud phase lifts this as an *optional* addition without regressing the free tier.
- Wear OS via `wear_plus` with a Kotlin fallback behind an interface (feasibility validated by an early spike).
- The iOS Health API requires a real device; the test strategy must therefore make the RPE-only path fully CI-testable.

3. Architectural design

3.1. Architectural style

PulseCoach implements **Clean Architecture** [8] with a **feature-first** package layout. Dependencies point inward: the presentation layer depends on the domain; the data layer implements domain interfaces; the domain depends on nothing. A distinct **AI engine** layer is kept in pure Dart (no Flutter imports) so that it is portable to an isolate and fully unit-testable without the widget framework. The **core** layer hosts cross-cutting infrastructure (DI, database, routing, theme, and, in the cloud phase, the cloud boundary).

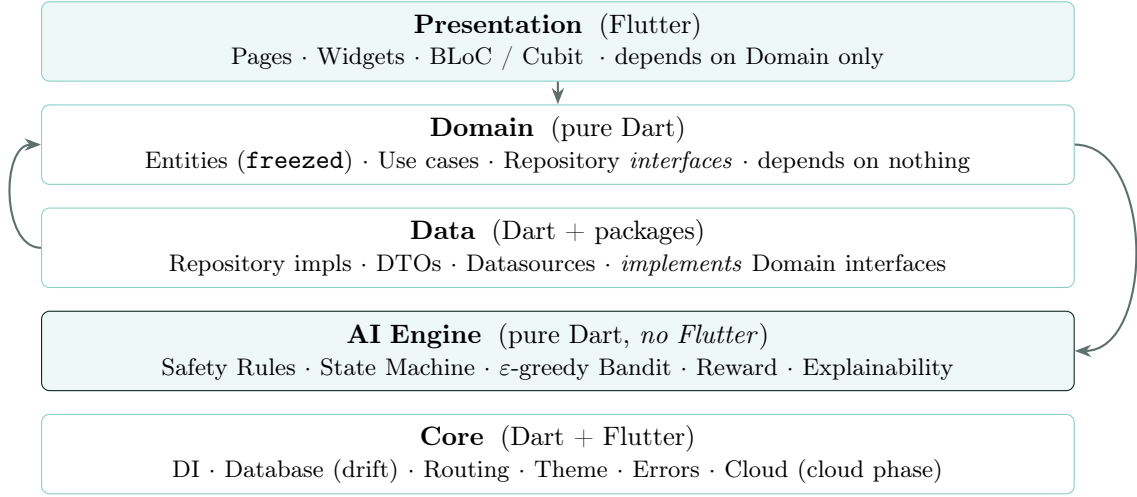


Figure 1: Layered architecture. Arrows denote the allowed direction of source dependency. The AI engine is consumed by domain use cases and is framework-free by construction.

The dependency rules are enforced by convention and code review rather than by a mechanical import-boundary linter; the import matrix in Table 4 is the contract. This differs from the determinism seams of Section 17.3, which are machine-enforced, and a dedicated dependency-gate in CI is a low-cost hardening left as future work. The residual risk, architectural erosion through an unintended import as the 419-file codebase grows, is accepted knowingly: its reach is bounded by the same code-review discipline that maintains the matrix today, and promoting the import contract to a mechanical lint would close it.

Table 4: Import (source-dependency) rules between layers.

From	May import	Must not import
Presentation	Domain, Core, Shared widgets	Data
Domain	(nothing)	Presentation, Data, Core, Flutter
Data	Domain, Core	Presentation
AI Engine	(nothing)	Flutter, Presentation, Data, Core
Core	Dart/Flutter packages only	Features, Domain, Data
Cloud (cloud phase, core/cloud/)	Dart/Flutter + supabase_flutter	Features, Domain, Data

3.2. Architectural styles and patterns

The architecture composes a few established styles, each adopted for a specific property rather than for its own sake:

- **Clean / layered architecture with dependency inversion** (Figure 1, Table 4): source dependencies point inward, so the domain and the AI engine compile without Flutter and are testable in isolation.
- **Feature-first modularization**: each feature is a self-contained `data/`/`domain/`/`presentation/` slice (Table 5), which localizes change and keeps the tests uniform.
- **Repository pattern with functional error boundaries**: repositories expose domain interfaces and return `Either<Failure, T>`, so exceptions never cross a layer and every failure is an explicit, testable value.
- **BLoC / Cubit unidirectional state**: past-tense events map to `frozen` state unions whose value equality drives minimal rebuilds; this is the presentation-layer analogue of MVVM.
- **Reactive, event-driven data flow**: the Drift database is the single source of truth and emits change streams, so components reconcile from those streams instead of signalling one another (Figure 8).
- **Offline-first, cache-first resilience** (Section 7.4): every external input has a cache and a fallback, so “no network / no sensor / no watch” is a normal path rather than an error.
- **Safety-constrained bandit** (Section 4): the decision core wraps a deterministic hard gate around an ϵ -greedy learner, so personalization can act only *inside* the permitted envelope.
- **Serverless client-server** (cloud phase): the optional backend is managed Postgres with Row-Level Security and Edge Functions; the client holds a single cloud boundary and the free core issues no server calls.

3.3. Module decomposition

Every feature follows an identical internal structure (`data/ · domain/ · presentation/`), which makes the codebase navigable and the tests uniform. Table 5 lists the feature modules and the requirements they own.

Table 5: Feature modules and requirement ownership.

Module	Requirements	Responsibility
onboarding	FR1–FR4	Disclaimer, animated onboarding, profile setup/edit
today / daily_plan	FR5–FR15	Daily plan orchestration, hero progression, explanations
session	FR16–FR25	Guided execution, timer, RPE capture, feedback loop
sessions_catalog	FR26–FR28	Exercise browsing, filtering, detail
progress	FR29–FR32	History timeline, charts, weekly goals
weather	FR33–FR36	Weather/AQI context provider
settings	FR48–FR52	Theme, privacy info, device/sync settings, export
shared/widgets	FR37–FR39	Responsive scaffold, adaptive navigation, shimmer
auth (<i>cloud phase</i>)	FR54– FR57, FR77	Sign-in, E2E backup/restore, account deletion
subscription (<i>cloud phase</i>)	FR58–FR62	IAP, entitlement gating, grandfathering
social (<i>cloud phase</i>)	FR63–FR76	Friends, feed, leaderboard, shared live sessions

Requirement FR53 (the single non-cloud item in the FR53–FR77 numbering block) is a purely client-side navigation return-affordance (from the drawer secondaries back to the primary shell) realized in `core/routing/app_router.dart`. It is owned by the Core layer rather than by any feature slice, which is why it does not appear as a feature row above; the appendix requirement matrix maps it there.

3.4. Technology stack and rationale

Each principal package is chosen to encode a specific design decision rather than for its own sake (Table 6).

Table 6: Principal packages and the design decision each encodes.

Concern	Choice	Rationale
State management	<code>flutter_bloc</code>	BLoC for domain flows, Cubit for UI-only state; explicit, testable emissions
Dependency injection	<code>get_it</code> + <code>injectable</code>	Compile-time DI; mockable boundaries everywhere
Persistence	<code>drift</code> + <code>drift_flutter</code>	Type-safe, reactive queries; in-memory DB for fast tests; isolate-friendly
Immutable models	<code>freezed</code> + <code>json_serializable</code>	Value equality drives correct BLoC rebuilds; <code>copyWith</code> ; serialization
Functional errors	<code>dartz</code> (<code>Either</code>)	Explicit, exhaustive error paths across layer boundaries
Navigation	<code>go_router</code>	Declarative shell + overlay routes; deep-linking for session resume
HTTP	<code>dio</code>	Interceptors for cache logic, fail-fast timeouts, logging
Charts / animation	<code>fl_chart</code> · <code>lottie</code>	Progress visualization; bundled offline animations
Cloud (cloud phase)	<code>supabase_flutter</code>	Postgres + RLS + Auth + Realtime in one EU-hosted provider
IAP (cloud phase)	<code>purchases_flutter</code>	Cross-platform entitlements, restore, grandfathering

3.5. Consistency rules

To keep a large, multi-author codebase coherent, the design fixes a small set of non-negotiable patterns:

- Repositories return `Either<Failure, T>`; exceptions never escape the data layer.

- Every BLoC state is a **frozen** union with at least **initial**, **loading**, **loaded**, **error**; events are past-tense, states are nouns.
- Loading renders a shimmer placeholder matching the content layout, not a spinner.
- External API responses are cached in drift with a **cachedAt** column; there is no in-memory-only cache.
- Expected degradation (sensor off, API down, watch disconnected) is handled silently and is never surfaced as an error state.
- AI-engine code contains no Flutter imports and is testable with plain **dart:test**.

3.6. Deployment view

Figure 1 shows how the code is layered, not where it runs. Figure 2 shows the runtime topology: which parts execute on the user’s device, which execute off it, and which links lie on the free/offline critical path rather than in the optional cloud phase. The boundary it makes explicit is the core/cloud split of NFR34. The decision engine and all core-phase state live on the device; the two core-phase integrations are anonymous, read-only REST calls that fall back to cache; and every authenticated link belongs to the cloud phase, reachable only from the cloud/social surfaces. An account-free user’s device therefore talks to nothing but the two public APIs.

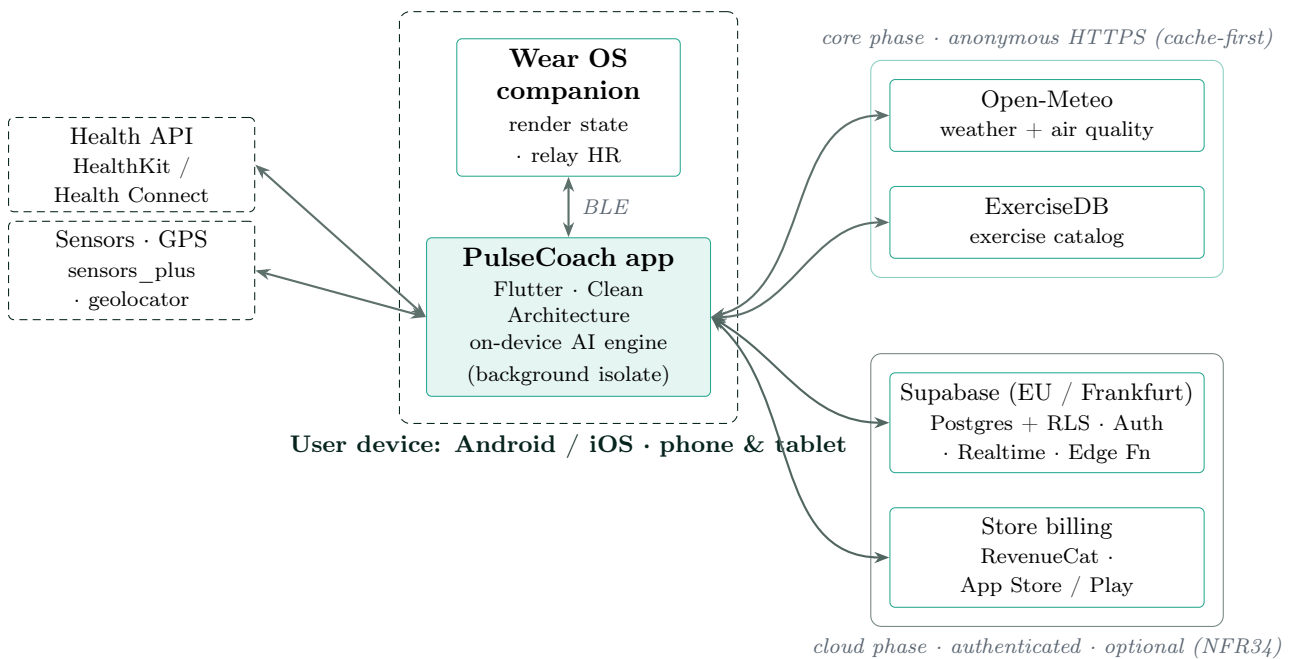


Figure 2: Deployment view: runtime topology. The decision engine and all core-phase state execute on the user’s device (centre); the on-device platform capabilities (left) are optional sensor/health inputs; the two core-phase integrations (upper right) are anonymous, cache-first REST calls, whereas every authenticated link (lower right) belongs to the optional cloud phase and issues only from the cloud/social surfaces (NFR34). Double-headed arrows denote request/response over HTTPS, or BLE for the watch.

4. The on-device decision engine

This is the core of the product and the part of the design treated in most depth here. The engine turns a snapshot of user and environment signals into a compact daily plan that stays inside the safety envelope, personalized to the user, plus a structured explanation. It is composed of four collaborating pure-Dart components arranged as a *pipeline with a hard safety gate*.

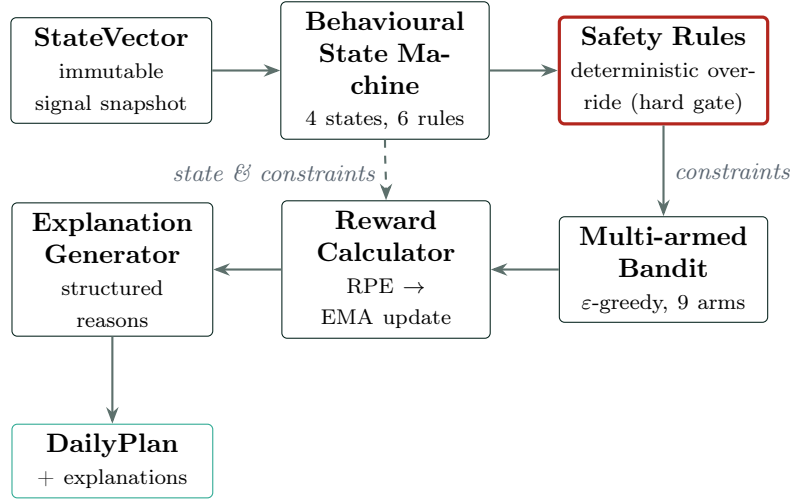


Figure 3: The decision pipeline. The Safety Rules component is a hard gate: the bandit only ever chooses among arms that safety has already permitted. Feedback (RPE $\rightarrow$ reward) closes the loop back into the bandit’s persisted weights.

4.1. The StateVector: the engine’s input

StateVector is an immutable, JSON-serializable snapshot of everything the engine considers. It is assembled at the use-case layer from sensors, weather, RPE history, and the user profile, then sent across the isolate boundary.

```

@freezed
abstract class StateVector with _$StateVector {
  const factory StateVector({
    required double? restingHR,           // health API (optional)
    required int? stepCount,              // health API (optional)
    required ActivityLevel? activityLevel,
    required List<int> rpeHistory,         // last N RPE values, most recent last
    required int missedSessions,
    required int streak,
    required AqiLevel aqiLevel,           // derived: high when AQI >= 100 (FR8)
    required double? temperature,
    required bool? precipitation,
    required UserProfile userProfile,
    required BehavioralState currentState,
  }) = _StateVector;
}

```

The nullable sensor fields make graceful degradation explicit: a new user with no wearable simply arrives with `restingHR == null`, and every downstream rule tolerates that.

4.2. Behavioural state machine

The state machine governs the *policy* regime. It is deterministic and stateless: each call recomputes the next state purely from the input snapshot. Four states model the user’s engagement/exertion arc; six *prioritized* rules define the transitions (Figure 4). Priority order matters: a disengagement signal (missed sessions) is checked before an exertion signal, because structural risk to the habit outranks a single hard workout.

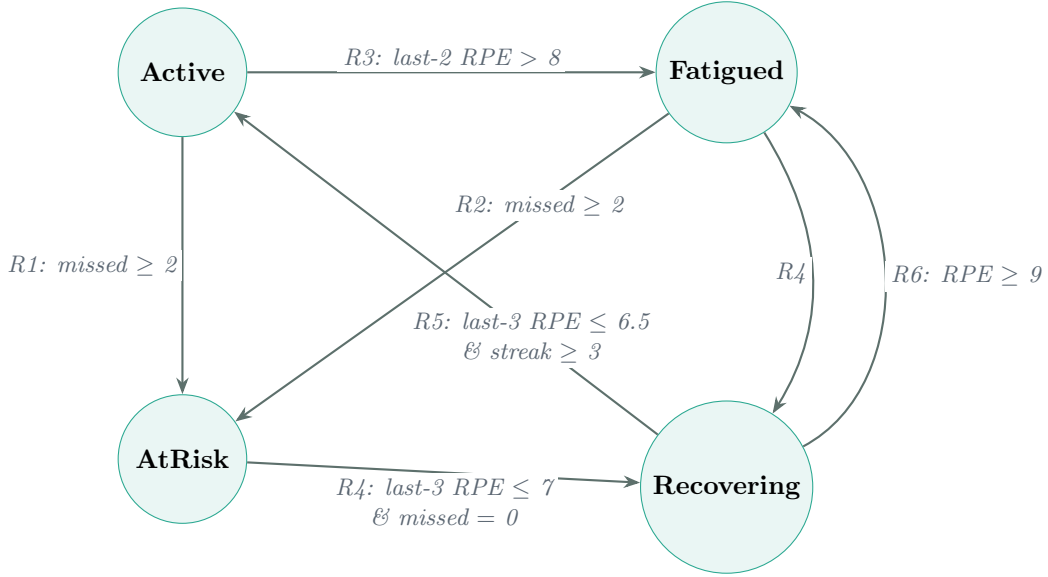


Figure 4: Behavioural state machine. Rules are evaluated top-to-bottom by priority (R1→R6); the first firing rule wins, otherwise the current state is retained.

Each state maps to a *constraint envelope* that caps intensity, session count, and outdoor eligibility (Table 7). This mapping (FR24) is centralized in a single method so that rules and tests reference one source of truth.

Table 7: State → constraint envelope (`constraintsForState`).

State	Max intensity	Max sessions	Outdoor
Active	none (no cap)	unbounded	allowed
Fatigued	medium	3	allowed
AtRisk	low	2	allowed
Recovering	low	2	allowed

4.3. Safety rules: the deterministic hard gate

The safety layer is the most important correctness boundary in the system. It merges constraints from three independent sources and always takes the *most restrictive* value; the bandit downstream can never widen them. It is exhaustive by construction: adding a new AQI or intensity level forces a compile error rather than a silent fail-open.

```

/// Merges constraints from three sources (most restrictive wins):
/// 1. Behavioral state (FR24): constraintsForState()
/// 2. Direct RPE signal (FR9): last-2-avg > 8 caps maxIntensity at medium
/// 3. Air quality (FR8): AqiLevel.high blocks all outdoor sessions
SafetyConstraints apply(StateVector v) {
    final base = _machine.constraintsForState(v.currentState);
    final maxIntensity = _mergedIntensity(base.maxIntensity, v.rpeHistory);
    final aqiAllowsOutdoor = switch (v.aqiLevel) { // exhaustive: fail-compile,
        AqiLevel.low => true, // never fail-open
        AqiLevel.high => false,
    };
    return SafetyConstraints(
        maxIntensity: maxIntensity,
        maxSessionCount: base.maxSessionCount,
        outdoorAllowed: aqiAllowsOutdoor && base.outdoorAllowed,
    );
}

```

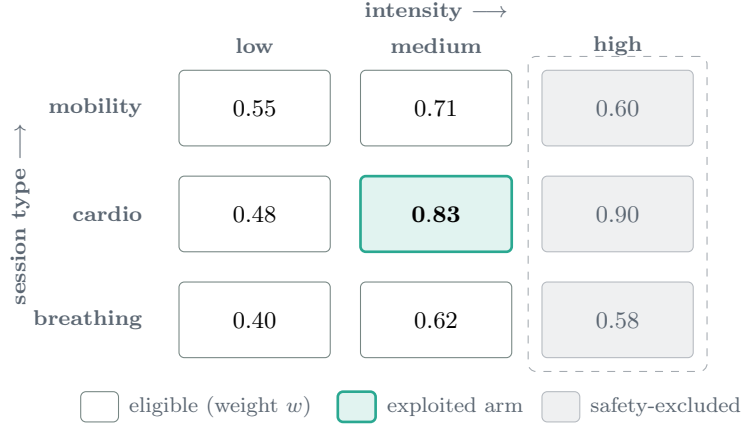


Figure 5: The nine arms as a session-type $\times$ intensity grid, at one decision step; cell values are the per-arm EMA weights of Eq. (2). The state machine and safety gate first mark the *eligible* set (here the high-intensity column is capped out, FR9); the ϵ -greedy learner then acts *only within it*: with probability $1 - \epsilon$ it *exploits* the highest-weight eligible arm (cardio $\cdot$ medium, $w=0.83$), and with probability $\epsilon = 0.2$ it *explores* a uniformly random eligible arm. The globally highest weight (cardio $\cdot$ high, $w=0.90$) is never selectable, because safety has excluded it: the learner can rank arms only *inside* the permitted envelope, never widen it. Selection over the plan is without replacement.

4.4. The bandit: ϵ -greedy over a safety-constrained action set

The learner is an ϵ -greedy multi-armed bandit [1, 10] over nine arms: three session types (mobility, cardio, breathing) $\times$ three intensities (low, medium, high). With probability ϵ (default 0.2) it *explores* a random eligible arm; otherwise it *exploits* the highest-weight eligible arm, with an alphabetical tiebreak for determinism. The eligible set is filtered by the safety constraints before selection, so the safety contract (FR9) is structurally guaranteed:

```

List<PlannedSession> selectSessions(
    StateVector v, SafetyConstraints c, BanditState state) {
    final eligible = _eligibleArms(c, state.armWeights); // safety filter first
    if (eligible.isEmpty) return [];
    final count = c.maxSessionCount.clamp(0, eligible.length);
    final remaining = List<String>.from(eligible.keys);
    final selected = <String>[];
    for (var i = 0; i < count && remaining.isNotEmpty; i++) {
        final arm = _random.nextDouble() < epsilon
            ? remaining[_random.nextInt(remaining.length)] // explore
            : _exploit(remaining, eligible);                // exploit
        selected.add(arm);
        remaining.remove(arm);                             // no duplicate arm
    }
    return selected.map((a) => _toPlannedSession(a, c)).toList();
}

```

Selection is *without replacement*, so a plan never repeats a session type $\times$ intensity. The bandit is stateless with respect to learning: it receives the persisted `BanditState` and returns an updated one, leaving persistence to the caller. Figure 5 shows one decision step of this arm space.

A note on terminology. User context (the `StateVector`) enters the decision by restricting the *feasible action set* (through the state machine and the safety gate), not by conditioning the arm-value estimate, which is a single scalar exponential moving average per arm (Eq. 2). This is therefore an ϵ -greedy multi-armed bandit over a context-constrained action space, not a contextual bandit in the LinUCB sense [6], where the reward model itself is a function of a context feature vector. Contextual bandits are in fact the established framework for just-in-time adaptive interventions in mobile health [9, 11], so PulseCoach’s single-weight value model is a deliberate simplification relative to that line of work; the more general pattern of constraining a bandit’s admissible actions by an external requirement rather than by its own value estimates also has mobile-health precedent [12]. The `StateVector` is threaded through the selection call for forward compatibility (a genuinely contextual value model is a natural extension point), but the learner as built does not condition its weights

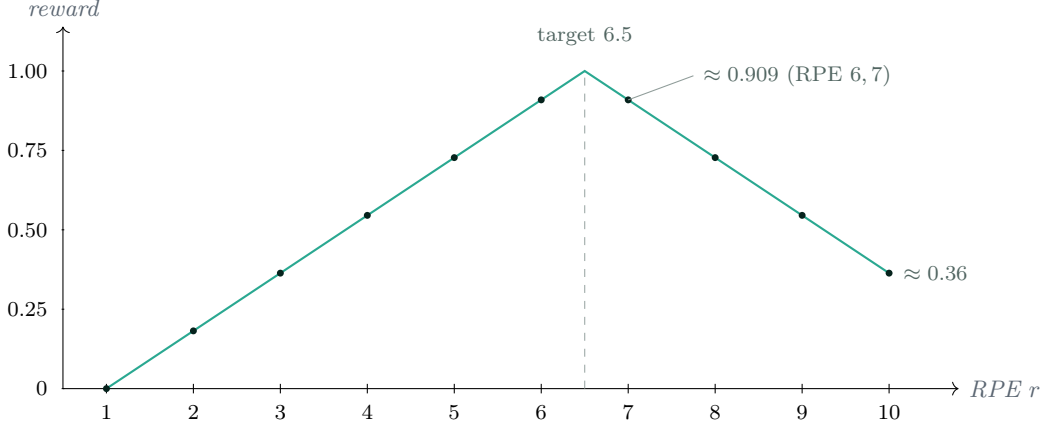


Figure 6: The reward function of Eq. (1), sampled at integer RPE (dots). The per-unit penalty is *identical* on both sides of the peak (slope magnitude $1/5.5 \approx 0.18$ per RPE point); the curve is asymmetric only because the target 6.5 is off-centre in $[1, 10]$, so the reachable low extreme (RPE 1, deviation 5.5) bottoms out at reward 0 whereas the reachable high extreme (RPE 10, deviation 3.5) floors at reward ≈ 0.36 .

on context. We name it carefully to avoid over-claiming the learner’s expressiveness. Injecting `Random` (rather than calling `Random()` directly) keeps exploration reproducible under test; see Section 17.3.

4.5. Reward calculator and the feedback loop

After a session the user reports an RPE [2]. The reward peaks at the target RPE of 6.5 and declines linearly with the absolute deviation from it, normalized by the larger half-range $5.5 = \max(|1 - 6.5|, |10 - 6.5|)$:

$$\text{reward}(r) = \max\left(0, 1 - \frac{|r - 6.5|}{5.5}\right), \quad r \in \{1, \dots, 10\}. \quad (1)$$

Because the peak (6.5) is not centred in the RPE range, the two tails are asymmetric (Figure 6): the reward reaches zero only at the low extreme (RPE 1, deviation 5.5) and falls to ≈ 0.36 at the high extreme (RPE 10, deviation 3.5). The per-unit penalty is the *same* in both directions ($1/5.5$ per RPE point); the asymmetry lies in the reachable endpoints, not the slope. This is intentional: because the reachable high extreme sits closer to the target than the reachable low extreme, the worst outcome for over-exertion (reward ≈ 0.36 at RPE 10) is less punishing than the worst outcome for complete disengagement (reward 0 at RPE 1), since a hard session is still an engaged session. The asymmetry never trades against safety: the reward only *ranks* arms that the hard gate of Section 4 has already deemed safe, so a residual weight on a vigorous arm can bias selection only *within* the permitted envelope; it can never widen it. The asymmetry is thus an engagement-shaping preference operating strictly downstream of the safety constraint, not a relaxation of it. Because RPE is integer-valued, the maximum attainable reward is ≈ 0.909 (at RPE 6 or 7). The chosen arm’s weight is then updated with an exponential moving average, learning rate $\alpha = 0.1$, which converges conservatively over roughly ten sessions:

$$w' = (1 - \alpha)w + \alpha \cdot \text{reward}(r). \quad (2)$$

Weights are *not* reset on state-machine transitions (FR25): learning accumulates continuously while the state machine only changes the policy regime. This is the closed loop of Figure 3: RPE feeds the reward, which nudges the weights, which shape the next day’s exploitation.

Parameter provenance. The engine’s constants are calibrated design defaults rather than empirically optimized values, chosen for conservative, interpretable behaviour on a small per-user data stream. The exploration rate $\varepsilon = 0.2$ keeps roughly one session in five exploratory, enough to escape a poor early estimate without letting randomness dominate the felt experience. It is held constant rather than annealed: the target is non-stationary (a user’s fitness, schedule, and preferences drift over months), so a small, steady openness to change serves better than a schedule that freezes exploration after early convergence and then cannot re-adapt. An annealed or state-dependent ε is left as an extension point. The learning rate $\alpha = 0.1$ gives the EMA an effective memory of $\approx 1/\alpha = 10$ sessions, so a single anomalous RPE cannot swing a weight sharply. The reward target of 6.5 sits at the centre of the “moderate” Borg band, expressing a preference for sustainable effort over

maximal exertion. The behavioural thresholds (last-2 RPE $> 8 \Rightarrow$ cap intensity; missed $\geq 2 \Rightarrow$ at-risk; streak ≥ 3 to return to Active) are heuristics grounded in the Borg RPE convention [2] and standard habit-formation reasoning, not in a clinical trial; they are deliberately conservative (they can only *reduce* prescribed load) and are gated behind the medical disclaimer accepted at onboarding. A systematic sensitivity analysis of these constants is future work.

Modelling scope and validity. Two properties of the learner should be stated explicitly, because the campaign of Section 17 verifies the engine’s *conformance to specification*, not the *efficacy* of its learned policy. First, reward is attributed to the chosen *arm*, whereas the reported RPE is a function of the arm *and* the user’s momentary state; since the value model carries a single scalar per arm with no state term, each weight is necessarily a state-averaged estimate. The state machine changes which arms are *feasible*, not how their value is estimated, so the two mechanisms are complementary rather than jointly conditioning a state-aware value function. Second, whether the learned policy produces better plans than a fixed, safety-gated one is an *efficacy* question the conformance campaign cannot answer on its own. We therefore ran an offline policy-evaluation simulation directly on the unmodified engine (Section 17.14): against a synthetic, heterogeneous population with within-user state variation it shows that on the tail of users whom any single fixed policy serves badly the learner beats the best hand-tuned static policy, while on a population centred near the reward target that fixed policy edges the shipped single-weight learner on the population average. The same harness shows that the two extensions named above (an annealed ε and per-behavioural-state value estimates) close that average-case gap monotonically, reaching parity-to-advantage; the extensions therefore rest on measured evidence, not speculation. A longitudinal field trial, as run for deployed activity-optimizing bandits in mobile health [7, 13], remains the only way to validate the synthetic model’s assumptions. We record all of this so that the modesty of the learner as built is not overstated and is not mistaken for a validated clinical policy.

4.6. Explainability

Every recommendation is accompanied by a structured, localized explanation derived from the decision context (which safety cap fired, which state the user is in, which environmental factor was decisive). Explanations are generated from stable *keys* rather than free text, which keeps them translatable (IT/EN) and testable; it is this projection of internal AI state that realizes the product’s central trust mechanism (FR13–FR15). The decision context also records whether a session was chosen by exploitation or by the ε -greedy exploration branch, so an exploratory pick can be labelled as such (“trying something a little different today”) instead of being presented as a confident recommendation, and the trust mechanism then also covers the roughly one-in-five exploratory session. Appendix D lists a representative slice of the explanation-key catalogue.

4.7. Missed-session decay

Engagement is not only reinforced, it also *decays*. A dedicated `MissedSessionsCalculator` derives the `missedSessions` and `streak` fields of the `StateVector` from the completion log and the device clock: consecutive days without a completed session increment `missedSessions`, which is exactly the signal that drives the state machine’s *active* $\rightarrow$ *atRisk* and *fatigued* $\rightarrow$ *atRisk* transitions (rules R1–R2). Because the calculation reads an injected clock, “two days missed” is testable to the hour rather than being wall-clock-dependent.

4.8. Group adaptation (cloud phase)

For co-located shared sessions, a deterministic `GroupConstraintResolver` (FR70) layers a group envelope on top of each participant’s individual core-phase safety cap: it takes the strictest intensity ceiling, the lowest fitness level, the *union* of movement exclusions, and the shortest available duration. It is pure Dart, stateless, and exhaustively unit-tested; it reuses the safety discipline rather than forming a parallel engine.

4.9. Putting it together: daily-plan generation

Algorithm 1 shows how the use-case layer orchestrates the components of Figure 3 into a single plan. The ordering matters: the state machine and the safety gate run before the learner, so the bandit only ever optimizes within a safe envelope. The entire body of Algorithm 1 executes on a background isolate.

Algorithm 1 On-device daily-plan generation (FR5–FR12).

```
1: input: StateVector  $v$  (signals snapshot), BanditState  $b$  (learned weights)
2: output: DailyPlan  $P$ , updated BanditState  $b'$ , explanations  $E$ 
3:  $t \leftarrow \text{StateMachine.evaluate}(v)$  {next behavioural state + transition key}
4:  $c \leftarrow \text{SafetyRules.apply}(v)$  {strictest of state / RPE / AQI caps: hard gate}
5:  $A \leftarrow \{ \text{arm} \in 9 \text{ arms} : \text{arm respects } c \}$  {eligible set}
6: if  $A = \emptyset$  then
7:   return empty plan {no safe option; never violate  $c$ }
8: end if
9:  $S \leftarrow \emptyset$ 
10: for  $i = 1 \dots \min(c.\text{maxSessionCount}, |A|)$  do
11:   if  $\text{rand}() < \varepsilon$  then
12:      $a \leftarrow \text{random arm from } A$  {explore}
13:   else
14:      $a \leftarrow \arg \max_{a \in A} b.\text{weight}(a)$  {exploit, alphabetical tiebreak}
15:   end if
16:    $S \leftarrow S \cup \{a\}; A \leftarrow A \setminus \{a\}$  {selection without replacement}
17: end for
18:  $P \leftarrow \text{buildPlan}(S, c); E \leftarrow \text{Explain}(t, c, v)$ 
19: persist  $t$ ; return  $(P, b, E)$  { $b$  updated later by the RPE feedback loop}
```

5. Concurrency and threading model

The threading model is explicit and enforced, because it directly serves NFR1 and NFR2:

- **Main isolate:** UI rendering and BLoC event handling only.
- **Compute isolate:** the entire AI pipeline runs via `compute()` on a background isolate. Inputs (`StateVector`, `BanditState`) and outputs (`DailyPlan`, explanations) are **frozen** value types transferable across the `SendPort` boundary without hand-written serialization.
- **Async I/O:** network (`dio`) and database (`drift`) use `async/await`; `drift` additionally exposes reactive `Streams` that the UI subscribes to.

The AI engine’s “no Flutter imports” rule makes isolate execution safe and also lets the engine be unit-tested with plain `dart:test`. The `AiEngine` abstract contract is mocked in tests and implemented by `AiEngineIsolate` in production.

6. Runtime view: dynamic behaviour

The viewpoints so far are *static* (the layer, module, and data structures) or *algorithmic* (Algorithm 1, which shows the engine *interior*). This section adds the *interaction* viewpoint of IEEE Std 1016 [4]: two sequence diagrams that show how the layers of Figure 1 collaborate *over time* for the two flows that carry the product’s essential dynamics. The first is daily-plan generation, which is where the architecture does the most work: it crosses the isolate boundary of the concurrency model and threads the graceful-degradation ladder while assembling its input. The second is the closing of the RPE feedback loop, which is deliberately *reactive*: no component signals another directly; the local database is the single source of truth and its change stream reconciles the UI. A third interaction, the cloud-phase co-located shared live session, is presented later in Section 14 (Figure 12), once its participants (the real-time gateway and the group resolver) have been introduced.

6.1. Daily-plan generation across the layers

Figure 7 traces one Today-screen open. The use case assembles the `StateVector` from the resilient data sources (each missing input silently degrades, per the graceful-degradation ladder of Section 7.4), hands it to the AI engine on a background isolate via `compute()`, persists the result, and returns an `Either<Failure, DailyPlan>` that the BLoC turns into a rendered state. The engine’s internal pipeline (state machine $\rightarrow$ safety gate $\rightarrow$ bandit $\rightarrow$ explanation) is the body of Algorithm 1; here it is a single opaque `compute()` call, which is the abstraction the isolate boundary buys.

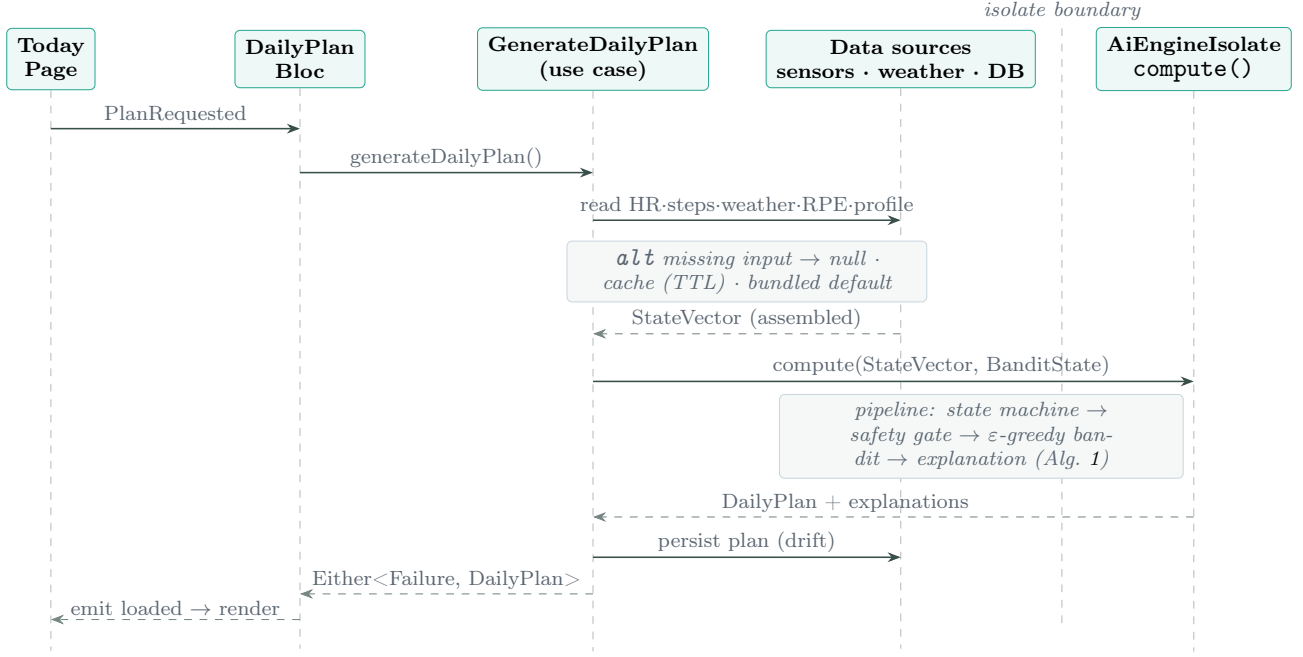


Figure 7: Daily-plan generation as a cross-layer interaction. The use case assembles a **StateVector** through the graceful-degradation ladder, the engine runs on a background isolate via `compute()` (crossing the boundary marked), and every layer return is an **Either**, so a data-source failure degrades exactly like a successful read. This is the runtime counterpart of the static layering in Figure 1 and the engine interior in Algorithm 1.

6.2. The reactive feedback loop

Figure 8 traces what happens when a session completes. The in-session cubit writes the completion to `session_logs` and feeds the RPE to the reward calculator, which updates and persists the bandit weights (Eq. (2)). Crucially, it does *not* then notify the Today screen: the `watchLogsForPlan` stream on the Drift table emits on the write, and `TodaySessionCubit`, subscribed to that stream, reconciles `completedIndices` and `heroIndex` from the database. The database is the single source of truth; the reactive edge (highlighted) removes cross-cubit signalling.

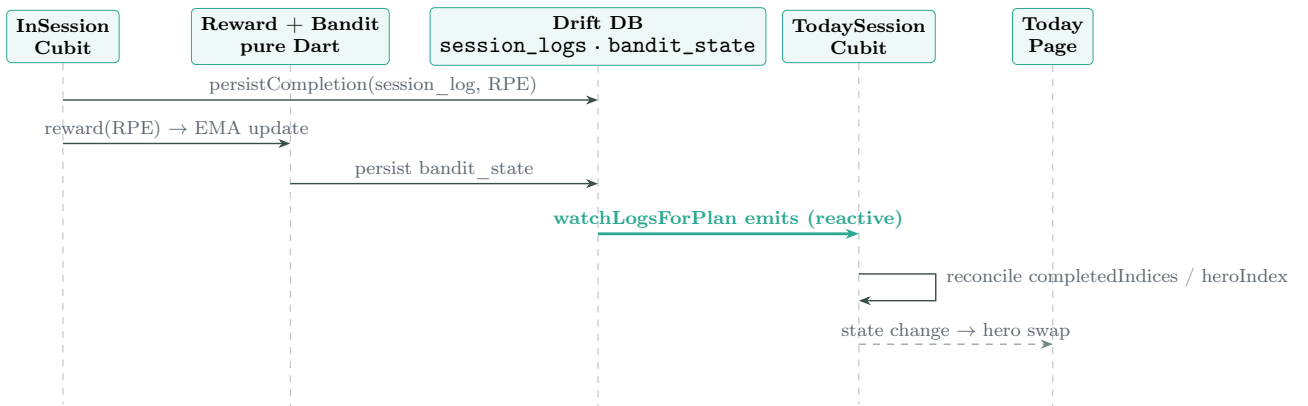


Figure 8: Session completion and the reactive feedback loop. The in-session cubit persists the completion and the updated bandit weights, but never signals the Today screen directly: the Drift `watchLogsForPlan` stream (highlighted) reconciles `TodaySessionCubit` from the database, which is the single source of truth. This is the runtime realization of the “no in-memory-only cache / DB is the truth” consistency rule.

7. Data design

7.1. Domain model

Figure 9 is the static view of the entities the decision engine reasons over and the associations between them. It reads as a cycle. The **Profile** and the recent history assemble a transient **StateVector**; the **StateVector** (with its **BehaviouralState**) safety-constrains which arms the **BanditState** may choose; the learner emits a **DailyPlan** of **Sessions**; each completed session produces a **SessionLog** with an **RpeFeedback**. Those logs close two loops: the RPE updates the bandit weights, and the completion history re-derives the **missedSessions** and **streak** fields of the next **StateVector**. Table 8 then gives the persistence mapping of these entities onto Drift tables.

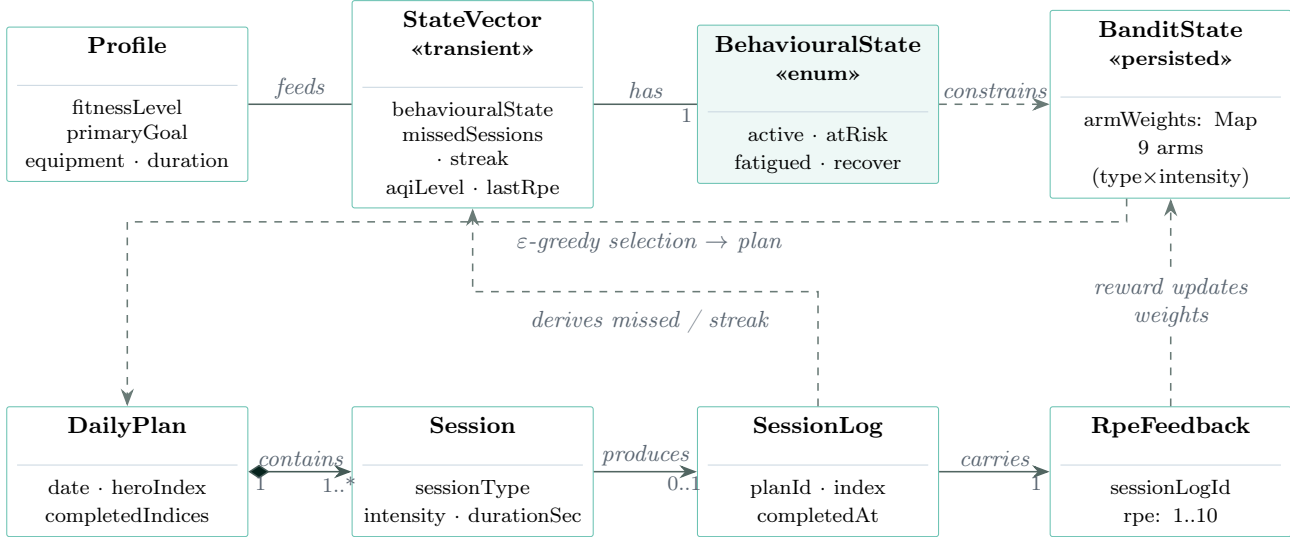


Figure 9: Domain model (class diagram). Solid lines are associations with multiplicities; the filled diamond marks composition (a **DailyPlan** is composed of its **Sessions**). Dashed arrows are the runtime dependencies that close the loop: the **StateVector** safety-constrains the **BanditState**’s feasible arms, the learner selects a **DailyPlan**, and each **SessionLog** with its **RpeFeedback** feeds back both the bandit weights and the next **StateVector**.

7.2. Local persistence (drift)

The local SQLite database, accessed through **drift**, is the single source of truth for the free core. Tables fall into three groups: *domain state* (profile, sessions, plans, RPE), *AI state* (bandit weights, behavioural state), and *caches* (weather, exercises) carrying a **cachedAt** timestamp for TTL logic. A dedicated **sync_queue** table backs deferred cloud writes.

Table 8: Drift tables and their role.

Table	Group	Content
user_profile	domain	Onboarding profile, goals, constraints
sessions	domain	Session definitions/history (legacy path)
session_logs	domain	Per-session completion (reactive source of truth)
daily_plans	domain	Persisted generated plans
rpe_feedback	domain	Captured RPE values feeding the reward loop
bandit_state	AI	Arm weights (the learned model)
behavioral_state	AI	Current behavioural state
weather_cache	cache	Open-Meteo response, TTL 1 h
exercise_cache	cache	ExerciseDB catalog, TTL 24 h
sync_queue	infra	Deferred cloud writes (cloud phase)

Schema evolution uses stepwise **onUpgrade** migrations so that learned bandit weights and session history survive

app updates (NFR15, NFR16). Completion is exposed as a Drift-native **Stream** (`watchLogsForPlan`), which lets the Today screen reconcile hero progression reactively from a single database source rather than through cross-cubit signalling.

7.3. Error model

Errors are values, not control flow. The **Failure** hierarchy is structurally comparable (equality by `runtimeType` + message), which matters because BLoC re-emission depends on value equality. Table 9 sets out the error contract layer by layer.

Table 9: Error handling by layer.

Layer	Contract
Remote datasource	Throws typed exceptions (<code>ServerException</code> , <code>CacheException</code> , <code>AuthException...</code>)
Repository	Catches; returns <code>Either<Failure, T></code>
Use case	Passes <code>Either</code> through; may combine results
BLoC	Pattern-matches <code>Either</code> ; emits <code>loaded</code> or <code>error</code>
UI	Renders from state; never catches exceptions

7.4. Graceful degradation

Every external read follows the same fallback ladder, which is the operational meaning of “offline-first”:

`primary (network/sensor) → cache (drift, within TTL) → bundled default → Failure`

For example, the exercise catalog falls back to a bundled set of 30 indoor-compatible exercises; weather falls back to indoor defaults when the cache is more than two hours stale. Expected degradation never produces an error UI.

8. External integrations and resilience

Table 10: External integrations and their resilience strategy.

Integration	Transport	Resilience
Open-Meteo (weather + AQI)	REST via <code>dio</code>	Cache in drift, TTL 1 h; indoor default if stale; city-level coordinates only
ExerciseDB (catalog)	REST via <code>dio</code>	Cache in drift, TTL 24 h; 30-exercise bundled fallback
Health API (HR, steps)	Platform channel	Optional; RPE-only path is fully functional without it
Accelerometer / GPS	<code>sensors_plus</code> / <code>geolocator</code>	Optional; GPS reduced to city level; momentary co-location boolean (cloud phase), never stored
Wear OS companion	<code>wear_plus</code> (+ Kotlin fallback)	Render state + relay HR; no on-watch logic; behind an interface; disconnect tolerated silently

Table 10 lists each external integration and its resilience strategy. The common rule is **fail-fast** + **cache-first**: outbound calls have short timeouts, failures fall to cache, and the UI is never blocked on the network. All integrations sit behind mockable interfaces (`dio` interceptors, datasource abstractions), so they remain testable in CI without hitting live services.

9. Component interfaces

This viewpoint makes the boundaries between components explicit. Internally, every cross-layer boundary is a narrow contract: repositories expose domain interfaces and return `Either<Failure, T>`, so no exception es-

capex the data layer, and the decision engine is reached only through the **AiEngine** abstract contract, which takes a **StateVector** and a **BanditState** and returns a **DailyPlan** with explanations across the isolate boundary (Sections 4 and 17). The boundaries that cross the network are specified below: the two anonymous external REST services of the core phase (Table 11) and the authenticated Supabase Edge Functions of the cloud phase (Table 12). Server-side scoring itself is performed by the SQL functions `award_session_points` / `award_shared_session_points`, verified with pgTAP (Section 17).

Table 11: External REST interfaces (core phase; anonymous GET via dio).

Endpoint	Request	Response and failure mode
Open-Meteo forecast GET /v1/forecast	latitude, longitude current=temperature_2m hourly= precipitation_probability timezone=auto	current.temperature_2m, hourly.precipitation_probability[0]. On failure: cached <code>weather_cache</code> (TTL 1h), else indoor default
Open-Meteo air quality GET /v1/air-quality	latitude, longitude current=european_aqi timezone=auto	current.european_aqi $\rightarrow$ <code>AqiLevel</code> . On failure: cache, else neutral AQI (no outdoor block)
ExerciseDB catalog GET/api/v1/ exercises	(none)	Exercise list (JSON). On failure: cached <code>exercise_cache</code> (TTL 24h), else 30-exercise bundled fallback

Table 12: Supabase Edge Function interfaces (cloud phase; Authorization: Bearer <jwt>).

Function	Request body	Behaviour and status codes
<code>delete_account_cascade</code>	(none)	Removes the Storage backup and deletes the auth user as the cascade root; 200 {success} · 401 · 500 {error} (FR77, NFR30)
<code>export_user_data</code>	(none)	200 JSON export (userId, email, timestamps, schemaVersion); 401
<code>score_shared_session</code>	{ sessionId }	Verifies caller $\in$ participants, computes <code>round(base $\times$ intensityWeight $\times$ 1.5)</code> ; 200 {award} · 400 · 401 · 403 · 500. The 1.5 shared-session bonus is a single server constant (FR75 kill switch)

Each Edge Function re-derives the caller identity from the JWT server-side and uses the service-role key only inside the function, so the client never holds elevated privileges. This is the network-boundary counterpart of the on-device layering rules of Table 4.

10. Optional cloud layer (cloud phase, delivered in v1.1.0): overview and invariants

The *cloud phase* (shipped together with the on-device *core phase* in application version 1.1.0, not as a separate later release; “core phase” and “cloud phase” denote development phases, not app versions) adds an *optional, additive* cloud layer for accounts, subscriptions, and social features. Its single governing invariant (NFR34) is that the account-free, offline core phase must never regress: every cloud call is reachable only from the cloud/social features, and the free path issues zero cloud calls. The backend is Supabase (managed Postgres) in the EU region; the client boundary lives entirely in `lib/core/cloud/`, the only place that imports `supabase_flutter`, and every cloud repository returns the same `Either<Failure, T>` as the core-phase data layer, so a cloud failure degrades exactly like an offline sensor.

Table 13: Cloud-phase decisions of record.

Concern	Decision
Backend	Supabase (managed Postgres), EU region (Frankfurt), NFR36
Authentication	Supabase Auth: email/password, Sign in with Apple, Google OIDC; tokens in <code>flutter_secure_storage</code> , NFR27
Privacy boundary	Postgres Row-Level Security enforces visibility tiers (private / friends-only) at the database, not the client, NFR29
Backup	Client-side E2E encryption with a user-held (passphrase-derived) key; the server stores only an opaque ciphertext blob, NFR28
Real-time	Supabase Realtime Broadcast (step/timer events) + Presence (lobby/drop-out); host authority; ≈ 1 s sync, NFR31
Billing	Platform IAP via RevenueCat; entitlement cache; grandfathering of pre-1.1.0 installs
Consent & erasure	Per-purpose <code>consents</code> rows (consent-before-write); <code>ON DELETE CASCADE</code> + Edge Function for export and erasure (≤ 30 d), NFR29, NFR30

10.1. Three-tier entitlement gating

All feature access is routed through a single `EntitlementGate` rather than scattered `if (isPro)` checks. It resolves three tiers (Table 14); a locked-feature tap raises a contextual, silent `ProUpsellSheet`, not a persistent paywall banner.

Table 14: The three capability tiers resolved by `EntitlementGate`.

Tier	Capabilities
Account-free	The complete core phase: daily planning, sessions, RPE loop, progress, offline, with no account and no cloud call
Signed-in free	+ E2E backup/restore, unique handle, leaderboard <i>ranking</i> view
Pro (active sub)	+ friends, activity feed, shared live sessions, scoring, full Progress history (pre-1.1.0 installs are grandfathered to full history for free)

11. Accounts, authentication and end-to-end backup (cloud phase)

The `auth` feature (FR54–FR57, FR77) provides an *optional* identity. Sign-in uses Supabase Auth with three providers: email/password, Sign in with Apple (store-mandated parity), and Google OIDC. Refresh tokens are persisted in the platform keychain via `flutter_secure_storage`. Signing in never becomes a prerequisite for a core flow; it only *unlocks* backup and the social/Pro surface.

11.1. End-to-end encrypted backup

The backup of the user’s profile, history, and (most sensitively) personalization state (bandit weights, behavioural state) is **end-to-end encrypted with a user-held key** (NFR28, FR57). The client derives a key from a user passphrase and encrypts the payload *before* upload; Supabase Storage receives only an opaque ciphertext blob plus non-secret metadata (`schemaVersion`, `createdAt`). There is no server-side key escrow by design: a lost passphrase means a lost backup, the price of a server that is structurally unable to read biometric-derived state. This is the cloud counterpart of the core-phase on-device privacy boundary: the biometric signal is never a readable column anywhere.

11.2. Account deletion

In-app account deletion (FR77, store-blocking under NFR32) is a domain use case backed by a `delete_account_cascade` Edge Function: the visible removal (feeds, leaderboard, friendships) is immediate, and the backend purge completes within the GDPR window (≤ 30 d, NFR30). Deletion failure returns an `error` state and never leaves partial local state, a property that is itself unit-tested at the P0 level.

12. Pro subscription and feature gating (cloud phase)

The `subscription` feature (FR58–FR62) integrates platform In-App Purchase through RevenueCat (`purchases_flutter`). Entitlements are validated server-side (an Edge Function checks the store receipt) and cached locally so that gating decisions are instant and survive brief offline periods. Two design points carry most of the complexity:

- **Grandfathering.** A durable `installCohort` flag (`pre_v2` / `post_v2`, code identifiers retained from before the “cloud phase” renaming) is written locally and mirrored to the cloud `profiles` row on sign-in, so that pre-1.1.0 users keep full Progress history for free even after a reinstall (FR61). The cohort must survive device changes for signed-in users, hence the cloud mirror.
- **Restore and management.** Purchase restore and cancellation route through the same entitlement resolver, so the `EntitlementGate` is always the single source of truth for what a given user may do (FR60).

13. Social graph, friends, feed and leaderboard (cloud phase)

The social surface is an *optional*, opt-in cloud-phase layer that sits on top of the offline, account-free coach and is never a precondition for it: the entire adaptive product (planning, safety, learning, explanation) functions with the social features switched off, and no cloud call is issued until the user deliberately enters this surface. It is subordinate to the coach by design, not a competing centre of gravity. Structurally, the `social` feature spans four sub-modules (FR63–FR76): friends, activity feed, comparison, and leaderboard. The cloud schema is a normalized Postgres model (`profiles`, `friendships`, `shared_sessions`, `session_participants`, `activity_feed`, `leaderboard_entries`, `consents`) with foreign keys arranged so that erasure cascades cleanly. Figure 10 shows the model: `profiles` is the hub, every other table references it, and each of those foreign keys carries `ON DELETE CASCADE`, so a single `profiles` deletion propagates a complete erasure across the social surface, the structural backbone of the GDPR erasure guarantee (NFR30).

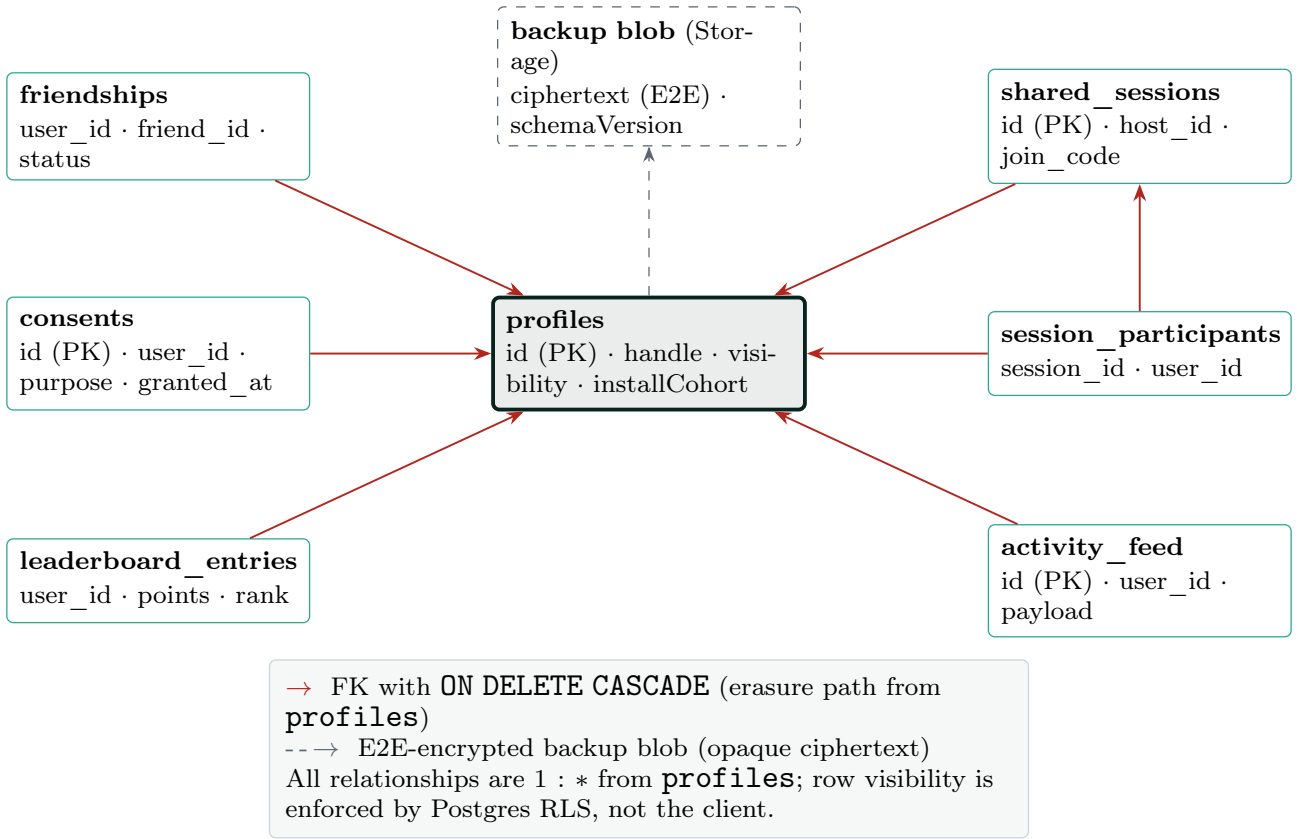


Figure 10: Cloud-phase entity-relationship model. **profiles** is the hub; every social table references it, and each such foreign key carries **ON DELETE CASCADE** (red), so one **profiles** deletion erases the user’s entire social footprint (NFR30). **session_participants** additionally references **shared_sessions**. The backup blob in Storage holds only opaque E2E ciphertext (NFR28); it is not a queryable table. Row visibility (**private**/**friends-only**) is enforced by Row-Level Security at the database, never by the client (NFR29).

13.1. Visibility is enforced at the database

The security boundary is **Row-Level Security**, not the client (NFR29). Each table carries RLS policies that implement the visibility tiers (**private** by default, **friends-only** on opt-in, never friend-of-friend) so that even a malicious client cannot read a row it is not entitled to. The client-side visibility checks exist only for UX (hiding a control) and are explicitly *not* the enforcement layer. Handle uniqueness and privacy-by-default (FR63) are likewise database-enforced.

13.2. Leaderboard and scoring

The friends-only leaderboard (FR74–FR76) is computed server-side. Point awards (for solo sessions and for a shared-session bonus) are performed by SQL functions (**award_session_points**, **award_shared_session_points**) with cap-clamping arithmetic, keeping the scoring logic off the client where it could be tampered with, and a rank-freeze mechanism avoids leaderboard churn. As discussed in Section 17, this SQL layer is verified with executable pgTAP database tests, not merely client-side passthrough tests.

14. Real-time shared sessions and group adaptation (cloud phase)

The most technically demanding cloud-phase feature is the **co-located shared live session** (FR68–FR73): several friends in the same room run one synchronized, group-adapted session. It is a net-new N -way real-time transport: it does *not* inherit the 1:1 phone↔watch bridge.

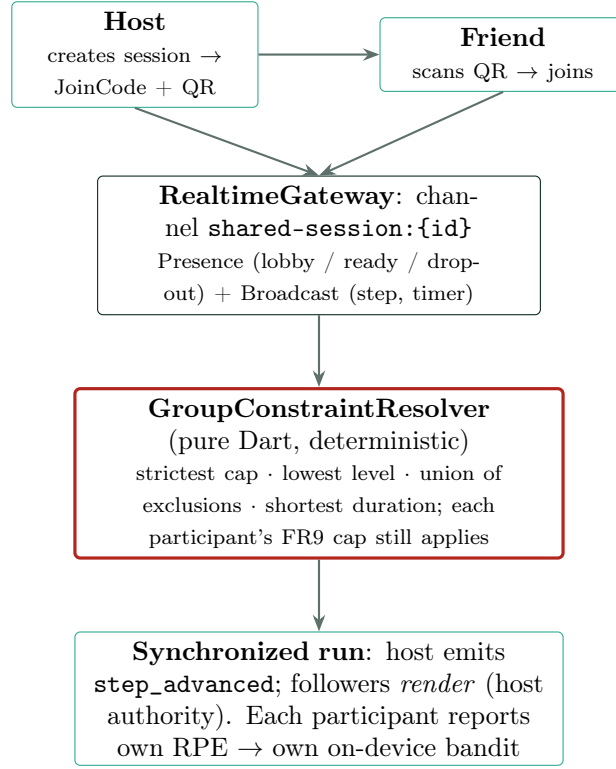


Figure 11: Co-located shared-session flow. Real-time transport (Presence + Broadcast) establishes the group; a deterministic resolver produces one safe group plan; the host is the sole authority for step advancement, which prevents divergence (NFR31).

Transport (FR71, NFR31).

Supabase Realtime *Broadcast* carries ephemeral step/timer events; *Presence* tracks the lobby and readiness. Nothing about the live session is persisted; it is transient by design. Sync is ≈ 1 s across N participants.

Host authority.

Exactly one participant is the host; only the host emits **step_advanced**, and followers render the received state; they never advance locally. This single rule makes N -way state convergent and drop-out tolerant: a participant leaving the Presence set does not interrupt the others.

Co-location (FR69, NFR33).

Joining performs a *momentary geolocator* read reduced to a co-located boolean; it is soft, non-blocking, and never stored. This is the one path that reads finer-than-city coordinates, because room-level proximity cannot be decided at city granularity; the NFR8 “city-level only” invariant is therefore scoped to the *outbound weather API path* (where rounded coordinates are all that ever leave the device), not to this transient on-device proximity check, whose fine reading is collapsed to a boolean in memory and never transmitted or persisted.

Group adaptation (FR70).

The **GroupConstraintResolver** of Section 4 produces one shared plan while each participant’s individual core-phase safety cap still applies, reusing the on-device engine rather than a second one.

Per-participant feedback (FR72).

Each participant’s RPE feeds *only* their own on-device bandit; there is no cross-user training, preserving the on-device personalization boundary even inside a group.

Figure 12 renders this flow as an interaction (the dynamic counterpart of the component view in Figure 11): the lobby forms through Presence, a single deterministic **GroupConstraintResolver** pass produces one safe group plan, the host is the sole authority for step advancement (the highlighted edge, which keeps the N participants convergent and drop-out tolerant), and feedback fans back to each participant’s *own* on-device bandit with no cross-user training.

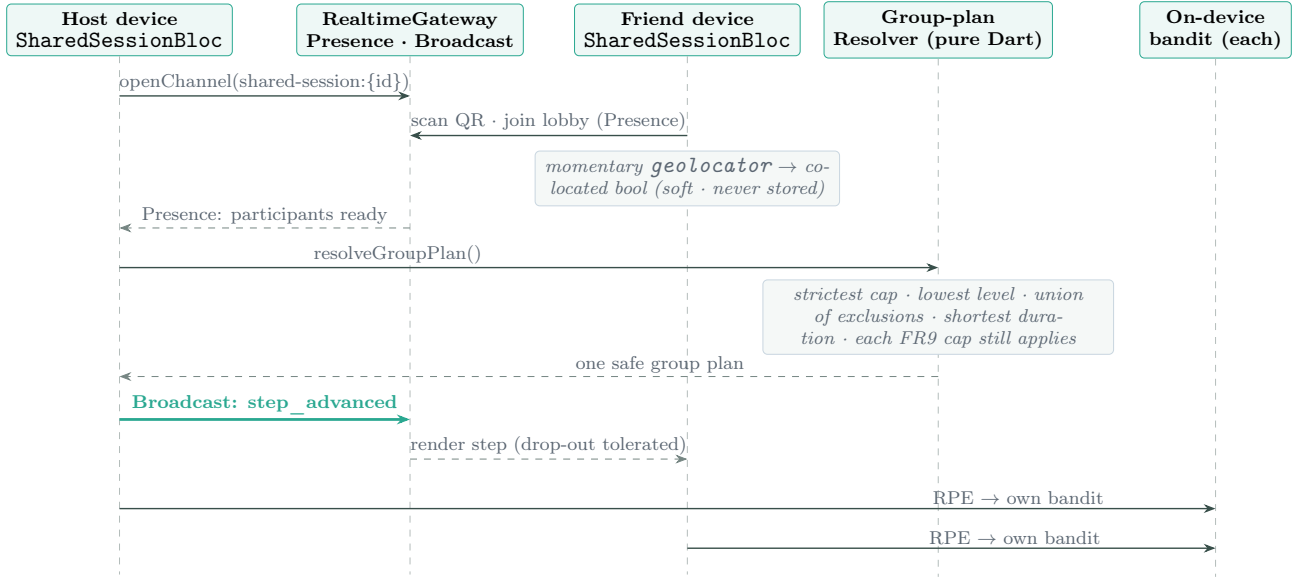


Figure 12: Co-located shared live session as an interaction (FR68–FR73). Presence forms the lobby; a single deterministic `GroupConstraintResolver` pass yields *one* safe group plan (the strictest of every participant’s envelope, each participant’s own FR9 safety cap still applying); the host is the sole authority for step advancement (highlighted), so followers only *render* and a drop-out never desynchronizes the others; and each participant’s RPE feeds *only* their own on-device bandit, with no cross-user training. This is the dynamic counterpart of the static component flow in Figure 11.

14.1. Privacy, consent and erasure across the cloud

The cloud layer preserves the core-phase privacy thesis under a social surface through four mechanisms working together: RLS as the enforced visibility boundary (NFR29); **per-purpose, unbundled consent** recorded in a `consents` table and checked *before* any personal-data write (NFR35); a GDPR export/erasure Edge Function with cascade delete (NFR30); and EU data residency (NFR36). Residual CLOUD-Act exposure (a US-incorporated BaaS) is mitigated by keeping biometric-derived state E2E-encrypted, minimizing server-side PII, and documenting a self-host-in-EU exit path.

15. Presentation and multi-device design

The layout decision is made once, at the scaffold level, via `LayoutBuilder` against a single 600 dp breakpoint. Below it (phone) the app uses a `BottomNavigationBar`; at or above it (tablet) a `NavigationRail` beside the shared content. Both expose the same four primary destinations: Sessions, Today, Progress, and Social. The Social destination is always present in the navigation, including for account-free users, but its social/Pro contents are gated by the `EntitlementGate` of Section 10; the free, offline core is unaffected because no cloud call is issued until the user enters that surface (NFR34). Components are shared across form factors; only the arrangement differs, and both portrait and landscape are supported. The Wear OS companion is a separate, thin companion target whose only roles are to *render* session state pushed from the phone and to *relay* heart rate back to it; it hosts no decision logic of its own, so all intelligence remains on the phone.

Beyond that single navigation breakpoint, the primary surfaces additionally re-arrange their *content* at wider widths, so the tablet experience is a genuinely distinct layout rather than a stretched phone. The sessions catalog switches, at a 720 dp content width, from a list whose rows open each exercise in a modal sheet to a *master-detail* split: a browsable list on the left and a persistent detail pane on the right that reflects the selected exercise. The Today screen similarly presents its hero session and the coming-up queue as two side-by-side panes at tablet width. At an 840 dp content width the progress dashboard lays its four charts out in a two-column grid instead of a single full-width column, and the profile-setup form is centred and width-capped so its segmented controls do not stretch edge to edge. Each transition is driven by a `LayoutBuilder` against the local content constraints, and *below* its breakpoint the widget subtree is exactly the phone layout, so no phone behaviour is altered. Because the switch keys on the available width rather than on a device flag, the same screens also adapt to *rotation*: a mid-size tablet that crosses the breakpoint when turned on its side moves between the single-column and the two-pane arrangements. Every one of these adaptive layouts is guarded by widget tests

that assert the correct arrangement at both a phone and a tablet surface size (Section 17). State management follows the BLoC/Cubit split (Table 15); every state is a **frozen** union whose value equality drives correct, minimal rebuilds.

Table 15: Representative BLoC / Cubit split.

Kind	Examples
BLoC (domain flows)	DailyPlanBloc, AuthBloc, SharedSessionBloc
Cubit (UI-only state)	InSessionCubit, ThemeCubit, ProgressCubit, LocaleCubit

Internationalization is wired through `gen_l10n` with ARB inputs; the language (IT/EN) is user-selectable at runtime via a `LocaleCubit` that drives `MaterialApp.locale` reactively.

16. Cross-cutting concerns

Privacy.

On-device AI, no biometric transmission, city-level location, GDPR Art.9 local processing [3]; in the cloud phase, RLS + per-purpose consent + E2E + EU residency preserve the privacy thesis under a social surface.

Explainability.

Every recommendation exposes its reason; the AI stack is designed to make its decision context available for display, debug, and export.

Graceful degradation.

A single, uniform fallback ladder governs all optional inputs, so “no wearable / no network / no watch” is a normal path, not an error.

Determinism.

All time- and randomness-dependent logic is parameterized (injected `Clock`, seeded `Random`), which is simultaneously a design property and the precondition for a reliable test campaign.

Consistency.

Fixed naming, structure, and error patterns keep a 419-file codebase navigable and reviewable.

17. Test campaign

17.1. Objectives and philosophy

The test campaign exists to protect a large, adaptive, multi-platform system against regression while remaining fast enough to run on every push. Its design follows three principles: a **layered test pyramid** (Figure 13: many cheap unit tests, fewer expensive integration tests); **risk-based prioritization** (test effort concentrated where a defect is most probable and most damaging); and **determinism** (every time- and randomness-dependent computation is parameterised, so tests are reproducible by construction rather than by retry).

What this campaign does and does not establish. The campaign establishes conformance to specification and regression safety: that the safety invariants hold, that the state machine and reward arithmetic behave as designed, and that no change silently breaks an existing acceptance criterion. It does not establish the efficacy of the adaptive core (whether the bandit’s learned policy produces subjectively better plans over time), which would require a longitudinal user study or a policy-evaluation simulation, both outside the scope of this document (see Section 17.17). The headline test count (1,421 cases) measures scale and breadth, not fault-detection power; the meaningful figure is acceptance-criterion coverage against the quality gate below, and branch- or mutation-level coverage is not separately reported.

17.2. Test architecture

Tests mirror the production layering. Table 16 shows the five levels, the concern each verifies, and the tooling.

Table 16: Test levels, scope, and tooling.

Level	Verifies	Tooling
Domain	AI logic (bandit, state machine, safety, reward), use cases, entities	<code>dart:test</code> (pure Dart)
Data	Repositories, caching, TTL, sync, datasources	<code>mockito</code> , <code>drift</code> in-memory
BLoC	State emissions and event handling	<code>bloc_test</code>
Widget	Rendering, responsiveness, rotation, reduce-motion, semantics	<code>flutter_test</code>
Integration	End-to-end flows (session, onboarding, plan generation)	<code>integration_test</code>

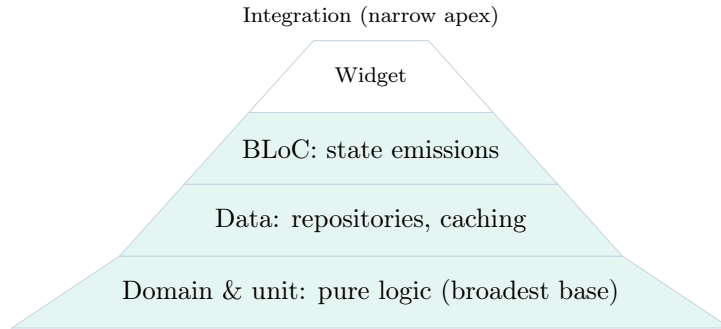


Figure 13: The test pyramid realized by the suite: the base is dominated by fast, deterministic pure-Dart domain tests; the expensive end-to-end apex is deliberately thin.

17.3. Determinism as a design requirement

Two architectural seams were mandated *before* test development because without them roughly sixty domain tests would be non-deterministic:

- **Injectable clock.** All time-dependent logic (TTL expiry, streaks, session timing) receives a `Clock` rather than calling `DateTime.now()`. Tests use a `FakeClock`.
- **Seeded randomness.** The bandit’s ϵ -greedy exploration receives an injected `Random`; tests use a fixed seed so exploration is reproducible.

This is why the code in Section 4 threads `Random` and clocks through constructors: the design and the testability are the same decision. These two seams remove the only *structural* sources of nondeterminism in the domain logic, and the full suite is executed on every push and pull request to `main` through the CI gate (Section 17) rather than relying on retries or quarantine. Residual UI-timing sensitivity at the widget and integration levels is contained by pumping to explicit settle points instead of fixed delays, so “deterministic” here is a property enforced at the seams.

17.4. Risk model driving coverage

The campaign is prioritized by an explicit risk register of eleven risks; the three high-priority ones (score ≥ 6) received mandated mitigations (excerpted in Table 17). Priorities P0–P3 are assigned to test scenarios accordingly, and coverage targets are set per priority.

Table 17: High-priority risks and their mitigation (excerpt of the register).

ID	Risk	Mitigation
R-001	<code>wear_plus</code> insufficient for Wear OS comms	Early spike; define <code>WearOsCommunicationService</code> interface regardless; phone tests mock it
R-002	iOS HealthKit untestable in CI (simulator limit)	Make RPE-only path fully CI-covered; schedule real-device sessions
R-003	Non-deterministic AI tests (<code>DateTime.now</code> , <code>Random</code>)	Inject <code>Clock</code> + seeded <code>Random</code> from day one

17.5. Traceability

Every automated test carries a stable identifier of the form {AREA}-{TYPE}-{NNN}, which ties it to a specific acceptance criterion and, through that criterion, to a requirement. A failing identifier therefore points directly at the requirement it guards. Table 18 illustrates the mapping style for a representative set of criteria.

Table 18: Requirement-to-test traceability (illustrative excerpt).

Acceptance criterion	Prio	Covering tests
Bandit never returns an arm the safety layer excluded	P0	SAFETY-001..00n, BANDIT-ELIG-*
Only applicable decision factors shown, never humidity	P1	FACTOR-001..005
Weather-fetch failure degrades, does not kill the plan	P1	WEATHER-CUBIT-002/004
Active-days: 29-day include / 30-day exclude boundary	P1	ACTIVE-DAYS-001..003
Inactivity timeout commits abandon on resume	P1	RECON-SVC-003/004, RECON-VIEW-002
Zero regressions (analyze + test)	P1	full-suite re-run

17.6. Quality gate

Coverage is measured against *acceptance criteria* (the test oracle), not lines: each criterion is classified FULL / PARTIAL / NONE and assigned a priority (Table 19). A subsystem passes the quality gate when the three rules of Table 20 all hold. As an example, the experience-polish subsystem resolved to **PASS** with P1 at 100% (23/23) and overall at 93% (40/43); the three residual partial items are structural-only claims (a negative visual assertion; platform-runtime notification behaviour) that cannot be closed without visual-regression or emulator/simulator lanes the project intentionally does not run.

Table 19: Priority classification of acceptance criteria.

Prio	Assigned to
P0	Safety-critical, data-integrity, or revenue-critical paths (e.g. the AI safety gate, auth-failure handling, IAP correctness). Coverage must be 100%.
P1	Core correctness and regression paths (most functional ACs). PASS target $\geq 90\%$.
P2	Presentation/UX polish, secondary paths, hard-to-automate visual claims.
P3	Cosmetic or record-only items.

Table 20: The three gate rules (all must hold for **PASS**).

Rule	Threshold	Meaning
P0 coverage	= 100%	no critical path may be unverified
P1 coverage	$\geq 90\%$ (target)	core correctness is proven
Overall coverage	$\geq 80\%$ of <i>assertable</i> ACs	the long tail is mostly covered

A small minority of acceptance criteria are *architecturally unassertable* in this project’s harness, whether negative visual claims (“no glow appears”), static-analysis facts, or platform-channel behaviour that needs a real device; each carries a documented rationale (Section 17.17). To keep this reclassification from becoming a way of making a subsystem pass by moving its failures out of the denominator, the classification is bound by an explicit rule: a criterion may be marked unassertable *only* if it falls into one of the three structural classes enumerated in Section 17.17 (negative visual claim, platform-runtime behaviour with no emulator/simulator lane, or real-device-only sensor path), and only P2/P3 criteria are eligible; P0 and P1 criteria are always assertable and must be covered outright. The classification is fixed at review time, before the gate is computed. The overall-coverage

rule is then evaluated over the *assertable* subset: an unassertable long tail cannot, on its own, fail a subsystem whose every testable criterion is covered, but neither can a genuine P0/P1 gap be reclassified away. The raw percentages in Table 21 are reported over all criteria for transparency; the gate decision itself is taken on the assertable subset, and Section 17.8 records two subsystems the gate actually blocked, which shows the rule can and does fail a subsystem rather than only confirming it.

17.7. Coverage results by subsystem

Coverage is tracked per subsystem and consolidated in Table 21. All eighteen subsystems ultimately reach **PASS**; two (marked †) failed an *initial* gate and were remediated before completion; discussed in Section 17.8. Where a cell reports a bare percentage rather than an explicit n/N fraction, the underlying acceptance-criterion count is maintained in the project’s coverage ledger; the gate decision itself is always computed from the full n/N , never from the rounded figure.

Table 21: Acceptance-criterion coverage by subsystem.

Subsystem	Overall	P1	Gate
Foundation & infrastructure	100% (64/64)	100%	PASS
Onboarding & profile	82.8% (24/29)	100%	PASS
Sensor integration	82% (9/11)	100%	PASS
Environmental context	91% (10/11)	100%	PASS
AI engine & daily planning	97% (30/31)	100%	PASS
Exercise catalog	100% (12/12)	100%	PASS
Today screen	91.9% (34/37)	96.7%	PASS
Internationalization	92.9% (13/14)	100%	PASS
In-session experience	95.8% (23/24)	100%	PASS†
Progress & history	75% (21/28)	100%	PASS
Responsive layout	85.7% (12/14)	100%	PASS
Wear OS companion	93.3% (14/15)	91.7%	PASS
Settings & extras	80% (16/20)	100%	PASS
Accounts & authentication	—‡ (7/12)→100% (12/12)	58%	PASS†
Pro subscription	80% (20/25)	91.7% (11/12)	PASS
Social graph & friends	91.4% (32/35)	100%	PASS
Leaderboard & scoring	100% (21/21)	100%	PASS
Experience polish	93% (40/43)	100%	PASS

‡Accounts & authentication: raw overall coverage was not separately tracked; the gate was decided on P0 (100%) and P1, whose initial FAIL at 58% and its remediation are discussed in Section 17.8.

Where *raw* overall coverage sits below 100% (e.g. progress/history at 75%), the uncovered items are, without exception, P2/P3 acceptance criteria that are *architecturally* unassertable in this project’s harness, whether animation timing, static-analysis facts, or platform-channel services (`share_plus`, `flutter_local_notifications`) that need a real device, each with a documented rationale confirmed by review. Because the overall-coverage gate rule is computed over the *assertable* subset (defined above), every subsystem clears the 80% threshold: progress/history, for instance, is 21/21 = 100% assertable once its seven documented, unassertable P2/P3 items are set aside. The raw figure is the transparent one; the assertable figure is the one the gate acts on. No P0 or P1 gap survives to close.

17.8. Two cases where the gate blocked completion

Twice the gate blocked a nominally complete subsystem and forced additional work.

- **In-session experience, initial FAIL.** A per-session persistence acceptance criterion was only partially covered; the subsystem was held until the reactive completion stream (`watchLogsForPlan`) and its tests closed the gap.
- **Accounts & authentication, initial FAIL at P1 58%.** P0 was clean (all twelve critical security/correctness criteria were fully covered), but five `AuthBloc` sign-in *event-handler* paths had complete data-layer coverage yet no `blocTest` verifying the state-emission sequence. The data layer was proven; the BLoC orchestration was not independently verified. The gate did not pass on “the pieces work” while the wiring was untested; the missing `blocTests` were added and the subsystem re-verified to **PASS**.

Both cases are the campaign behaving as intended: a P1 orchestration gap is a real gap even when every underlying unit is green.

17.9. Backend and database testing

The cloud-phase logic is not tested only from the client. The scoring and visibility rules that live in Postgres are verified with **executable pgTAP database tests** run against a real local Supabase instance (`supabase start && supabase test db`). For the leaderboard/scoring subsystem, three pgTAP files assert cap-clamp arithmetic for `award_session_points / award_shared_session_points`, active-only shared-session scoring, and a *friends-only negative-path* on `get_friends_leaderboard()`; all 18 pgTAP subtests pass. Running them surfaced two real database bugs, including a schema-wide missing-GRANT defect that the Dart client tests could never have caught because they mock the transport. This is the database analogue of the “orchestration gap” lesson: RLS and SQL functions are a security boundary, so they are tested where they execute.

17.10. Test-suite growth

The suite grew monotonically with the product, from a 68-test foundation to 1,409 passing cases at the v1.1.0 release and 1,415 after the post-release maintenance batch of Section 17.11, and 1,420 after a subsequent tablet adaptive-layout increment (Section 15) that added the master-detail catalog, the two-column progress grid, and the width-capped profile form, each with its guarding widget tests. The final development phase alone illustrates the steady, regression-guarded cadence: the count moved 1,331 → 1,346 → 1,355 → 1,363 → 1,366 → 1,388 → 1,409 → 1,415 → 1,420 across successive increments, each gated by “zero regressions” on the full suite plus a clean `flutter analyze`.

17.11. Post-release hardening: the v1.1.0 maintenance batch

On-device use after the v1.1.0 release surfaced a cluster of six wiring and platform-integration defects that unit-level conformance could not have predicted, since each lived at a composition boundary rather than inside a single unit. They were fixed as one maintenance merge, and every fix was landed with a guarding test, taking the suite from 1,409 to 1,415 passing cases with no new test files (the additions extend existing DI, health-datasource, and Social widget suites).

The two most instructive were *dependency-injection lifetime* bugs, invisible to any test that constructs its subject directly. First, `TodaySessionCubit` could not be resolved from the container at all: its optional `DateTime Function()` clock parameter (present so tests can inject a fixed “now”) was one **injectable** tried and failed to satisfy from the graph; annotating it `@ignoreParam` lets codegen skip the un-resolvable seam while preserving the test hook. Second, and more consequential, `AuthBloc` was registered as a *factory* rather than a singleton, so every injected consumer, notably `BackupBloc`, received its own fresh, always-unauthenticated instance; “Backup now” therefore failed with “Not authenticated” even while the user was signed in. Promoting it to `@lazySingleton` makes one shared auth state visible to all consumers. These are now guarded, respectively, by a container-resolution test and a singleton-identity test (two resolutions returning the *same* instance) in the DI suite: neither could be expressed at the level of a directly-constructed object, which is exactly why the original gate did not catch them.

The remaining fixes are similar boundary corrections, each with a matching test: the signed-out Social screen now renders an explicit sign-in call-to-action instead of silently reusing the loading shimmer (a new widget test asserts the Pro-but-unauthenticated banner and the absence of the tab bar), and the best-effort deep link into the Health Connect permission screen is covered by unit tests that assert the native-channel invocation and that a thrown `SecurityException/MissingPluginException` is swallowed so the caller is never disrupted. The host-Activity and demo-seed changes in the same batch are platform-runtime and debug-only surfaces

respectively, verified manually rather than in the harness.

One detail shows the full-suite gate at work. Promoting `AuthBloc` into an ancestor gate for the Social screen’s Pro content changed a widget’s implicit contract, and four pre-existing Social widget tests that had never supplied an `AuthBloc` broke immediately under the “zero regressions” rule; they were updated in lockstep to provide an authenticated ancestor. The blast radius of a one-line DI-annotation change was not obvious from the diff; the full-suite gate made it observable before merge.

17.12. Campaign results

The full suite was executed for this report; Appendix C gives the exact commands that reproduce it. Table 22 reports the headline numbers and Table 23 the distribution of test files across levels.

Table 22: Measured campaign outcome (full-suite execution for this report).

Metric	Value
Automated test files	183
Executed test cases	1,421
Passed	1,420
Skipped	1
Failed	0
Static analysis (<code>flutter analyze lib test</code>)	0 issues
Quality-gate outcome (all 18 subsystems)	PASS

Table 23: Test-file distribution across levels (approximate, by directory).

Level	Files	Focus
Widget	51	Rendering, responsiveness, semantics, reduce-motion
BLoC / Cubit	38	State emissions (incl. auth, shared session)
Data	34	Repositories, datasources, caching (incl. cloud/social)
Domain	22	AI engine, use cases, group constraints
Core	16	DAOs, DB, utils, errors, cloud client
Unit / feature / integration / 110n	22	Services, feature slices, end-to-end, localization

17.13. What the AI-engine tests assert

Because the decision engine is the product’s differentiator, its tests are correspondingly exhaustive and, thanks to determinism, exact rather than statistical:

- **Safety rules:** the strictest-wins merge across state, RPE, and AQI is verified for every combination; the bandit is asserted never to return an arm the safety layer excluded.
- **State machine:** each of the six prioritized rules is tested at and around its boundary (e.g. last-3 RPE exactly 7.0 vs. 7.1), plus the priority ordering itself.
- **Bandit:** with a seeded `Random`, exploration/exploitation decisions are checked deterministically; without-replacement selection and the alphabetical exploit tiebreak are pinned.
- **Reward:** Equation (1) is checked at the target, extremes, and the integer-RPE maximum (≈ 0.909), and the EMA update of Equation (2) is verified to be immutable and convergent.
- **Group resolver:** the min/lowest/union/shortest aggregation is tested exhaustively over participant sets.

17.14. Offline policy evaluation: does the learner earn its complexity?

The conformance tests above prove the engine behaves as specified; they cannot show that learning is worthwhile, nor whether the two extensions flagged in Section 4 (an annealed ε ; per-behavioural-state value estimates) would help. To settle both questions we built a small, deterministic policy-evaluation harness (`tool/policy_evaluation_sim.dart`) that drives the *unmodified* production classes (`BanditEngine`, `RewardCalculator`, `BanditState`) against a synthetic population of 500 users over a 60-session horizon. It is a standalone tool, not a widget test, and is not counted among the 1,421 automated cases.

Method. Each synthetic user has a latent fitness, and each session places the user in one of two load states (*normal* or *strained*), where a strained session makes every arm feel harder (its perceived RPE shifts up). The arm whose expected RPE lands nearest the target (6.5) therefore depends on *both* the user and the current state: this is precisely the arm $\times$ state interaction behind the reward-attribution confound of Section 4, under which a single scalar weight per arm can only ever learn a state-*averaged* value. Observed RPE is a Gaussian draw around the arm’s expected value; reward is the engine’s own Eq. (1). A session is counted in the *good-band* column of Table 24 when its observed RPE lands within ± 1.5 of the target ($\text{RPE} \in [5, 8]$), a coarser readability companion to the mean reward. Safety is held at a fixed permissive envelope (one session/day, all nine arms eligible) so it clamps every policy identically and the comparison isolates the learner. *Common random numbers* are used (for each (user, day) the load state and the noise shock are drawn once and shared across policies), so differences reflect the policies, not the draw. Five policies run on that shared environment: a **hand-tuned fixed** policy (always `mobility_medium`, the population-central arm); the **shipped learner** ($\varepsilon = 0.2$, one state-averaged value model); the same **with annealed** ε ; the same **with annealed** ε **and per-state value estimates** (a separate `BanditState` per load state, the behavioural-state analogue the app already computes); and an **oracle** upper bound. All are reproducible from the seed.

Table 24: Offline policy evaluation (seed 42; mean reward on the converged window, higher is better). Values are *point estimates* over the converged window; no dispersion is reported, so the sub-1% separations between adjacent rungs are within run-to-run noise and should be read as *parity*, not as a resolved ordering. Across seeds 7/42/100 only the qualitative structure is stable: fixed and the two extensions cluster together near the top, the shipped learner trails them on this population by $\approx 2\text{--}3\%$, and the oracle sits well above all four.

Policy	Mean reward	Good-band	vs. fixed
Fixed <code>mobility_medium</code>	0.705	66.3%	—
Shipped learner ($\varepsilon = 0.2$)	0.682	62.9%	−3.3%
+ annealed ε	0.693	65.0%	−1.8%
+ annealed ε + per-state	0.704	67.9%	−0.1%
Oracle (upper bound)	0.822	91.7%	+16.6%

Findings. Three results emerge; we state the least favourable first. *(i) On the population average, a well-centred fixed policy is hard to beat.* Because the population is centred on the reward target and the state varies within each user, the shipped single-weight learner trails the static `mobility_medium` by $\approx 2\text{--}3\%$: it pays for its permanent 20% exploration, its optimistic weight prior (1.0, above the attainable-reward ceiling of 0.909, which biases every early estimate upward until the EMA unwinds it), and its inability to separate the two load states in one averaged weight. Lowering the prior to ≤ 0.909 is a one-line change that joins the two extensions above as a candidate improvement; we leave it grouped with them rather than silently patching the shipped constant. *(ii) The two proposed extensions close that gap.* Annealing ε alone recovers most of it (to $\approx -0.4\%$ across seeds), and adding per-state value estimates lands within noise of the fixed policy on the average (-0.1% , $+1.6\%$, $+1.1\%$ at seeds 42/7/100) while pushing further toward the oracle. Because these separations are below the run-to-run noise floor, the defensible claim is *parity*, not a strict win: the extensions erase the shipped learner’s measurable deficit rather than overturning the fixed policy. They nonetheless confirm that the design notes of Section 4 identified the right levers. *(iii) Where personalization matters, learning wins.* On the atypical tail (the $\approx 40\%$ of users whose optimum sits far from the population centre, whom a fixed policy serves badly), even the shipped learner beats the fixed policy (≈ 0.68 vs. ≈ 0.63), and the per-state variant extends the lead (≈ 0.69). This tail effect is the clearest signal in the study, but its *magnitude in the field depends on how many real users are off-centre*, which the synthetic population fixes by construction rather than measures; the $\approx 40\%$ figure is therefore a modelling assumption, not an empirical population estimate. The headline is therefore that the shipped learner already wins for the users personalization exists to serve, while the two named extensions convert its average-case deficit into parity-or-advantage. The current bandit does not dominate every alternative; what the simulation gives is a measured path from that deficit to parity-and-beyond. A longitudinal field trial remains the only way to validate the synthetic model’s assumptions.

17.15. Tested configurations

The automated campaign runs headless on the Dart VM and is device-independent by construction (injected `Clock`, seeded `Random`, in-memory `drift`). It cannot reach the platform-runtime surface: real sensors, process death, and the tablet form factor. Those were exercised on the configurations of Table 25.

Table 25: Tested configurations.

Target	Class	Platform	Surface exercised
Physical Android phone	Phone	Android 8.0 (API 26), 1080×1920	Disclaimer → onboarding → shell; force-stop/relaunch persistence; four-tab navigation; catalog browse/filter
iPad Pro 13 inch	Tablet	iPadOS (simulator)	Master-detail catalog; two-pane Today; centred, width-capped profile form
CI runner	—	flutter test (headless)	Full automated suite: 1,421 cases across 183 files; flutter analyze at 0 issues

Every target passed its exercised surface with no functional defect. A dedicated Android emulator lane and an automated performance-measurement lane are the two configurations the project does not yet run; both are recorded as hardening opportunities in Section 17.17.

17.16. Manual on-device verification

The automated suite is complemented by manual smoke verification on a physical Android device, which covers exactly the platform-runtime surface the CI harness cannot reach. The build installed and launched cleanly; the medical-disclaimer gate, the animated onboarding carousel, and the profile-setup form all rendered and advanced correctly, and a full onboarding pass (goal, level, duration, and equipment selections enabling *Start My Plan*) navigated into the main shell. The four primary tabs (Sessions, Today, Progress, Social) and the navigation drawer were exercised without crash, the Today screen rendered its full adaptive layout, and the exercise catalog browsed and filtered. Critically for the persistence design, a force-stop followed by relaunch after completed onboarding returned directly to the Today screen, confirming the onboarding/profile persistence and the router-redirect logic end-to-end on real hardware. The tablet presentation was additionally exercised on the iPad Pro 13-inch simulator, where the sessions catalog rendered its master-detail split, the Today screen its two-pane arrangement, and the profile-setup form its centred, width-capped column. These checks are manual and unautomated by nature; they are recorded here for completeness, not counted among the 1,421 automated cases.

17.17. Known limitations of the campaign

Three classes of check are out of automated scope, each for a structural reason:

1. **Negative visual claims** (e.g. “no glow/confetti appears”): not expressible without a visual-regression harness; covered by code review.
2. **Platform-runtime behaviour** (Android ongoing-notification dismissal, iOS one-shot semantics): verified structurally (correct platform-details construction) since the project runs no emulator/simulator lane.
3. **iOS HealthKit**: requires a real device; mitigated by making the RPE-only fallback fully CI-covered and scheduling real-device sessions.

None of these corresponds to a known functional defect; each is a documented trade-off consistent with the academic scope.

Distinct from these three *structural* exclusions, the quantitative performance targets, namely NFR1 (plan <30 s), NFR2 (60 fps under AI computation), NFR4 (haptic <200 ms), and NFR6 (cold start <3 s), are at present verified by construction (off-UI-thread isolate execution via `compute()`, no network on the critical path, direct platform-channel haptics, and lazy DI) and by the manual on-device observation reported above (no perceptible jank or startup delay), but not by an automated performance-measurement lane. Unlike the three items above this is a hardening opportunity rather than a structural impossibility: adding trace-based latency and frame-time budgets to CI would convert these targets from argued-and-observed to measured, and is the natural next step for the campaign.

18. Conclusions

PulseCoach’s design is organized around a single idea: put a compact, safety-gated decision engine on the device, one that explains every recommendation it makes, and build a resilient, privacy-first, test-driven application around it. Clean Architecture with strict layer boundaries keeps the 419-file codebase coherent; the pure-Dart

AI engine (a safety-gated pipeline of state machine, deterministic override, and ε -greedy bandit) is, because it is framework-free and deterministic, the most thoroughly tested component. The engine’s differentiator is the *safety-gated, explainable envelope* rather than the learner alone: the offline evaluation of Section 17.14 reports openly that, as shipped, the single-weight bandit trails a well-centred fixed policy on the population average and is useful mainly on the off-centre tail, with two identified extensions closing the average-case gap to parity. The adaptive core is therefore a modest and still-unproven benefit layered on a safety and explainability contract that holds whether or not the bandit learns well. The optional cloud phase extends the product into accounts, subscriptions, and real-time social sessions without compromising the offline, account-free core.

The verification matches the design intent. A layered campaign of **1,420 passing automated tests across 183 files** — risk-prioritized and deterministic by construction — with a clean static-analysis baseline and a per-subsystem quality gate resolving to **PASS**, provides continuous evidence that the design conforms to its specification. The few checks left outside automation are named explicitly and justified: structurally where they are impossible in this harness, and as a stated hardening path (performance instrumentation) where they are not. Together, the design and the campaign demonstrate a system that is feature-complete across its 82 functional and 39 non-functional requirements and engineered to remain correct as it evolves.

References

- [1] Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2–3):235–256, 2002. DOI: <https://doi.org/10.1023/A:1013689704352>.
- [2] Gunnar Borg. *Borg’s Perceived Exertion and Pain Scales*. Human Kinetics, 1998. ISBN 978-0-88011-623-7.
- [3] European Parliament and Council. Regulation (eu) 2016/679 (general data protection regulation), 2016. Article 9 — processing of special categories of personal data. Available at: <https://eur-lex.europa.eu/eli/reg/2016/679/oj>.
- [4] IEEE. IEEE std 1016-2009 — IEEE standard for information technology—systems design—software design descriptions. Technical report, Institute of Electrical and Electronics Engineers, 2009. DOI: <https://doi.org/10.1109/IEEESTD.2009.5167255>.
- [5] ISO/IEC/IEEE. ISO/IEC/IEEE 29119 — software and systems engineering — software testing. Technical report, International Organization for Standardization, 2022. General concepts (Part 1:2022). Available at: <https://www.iso.org/standard/81291.html>.
- [6] Lihong Li, Wei Chu, John Langford, and Robert E. Schapire. A contextual-bandit approach to personalized news article recommendation. In *Proceedings of the 19th International Conference on World Wide Web (WWW)*, pages 661–670, 2010. DOI: <https://doi.org/10.1145/1772690.1772758>.
- [7] Peng Liao, Kristjan Greenewald, Predrag Klasnja, and Susan Murphy. Personalized HeartSteps: A reinforcement learning algorithm for optimizing physical activity. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies (IMWUT)*, 4(1):1–22, 2020. Article 18. DOI: <https://doi.org/10.1145/3381007>.
- [8] Robert C. Martin. *Clean Architecture: A Craftsman’s Guide to Software Structure and Design*. Prentice Hall, 2017. ISBN 978-0-13-449416-6.
- [9] Inbal Nahum-Shani, Shawna N. Smith, Bonnie J. Spring, Linda M. Collins, Katie Witkiewitz, Ambuj Tewari, and Susan A. Murphy. Just-in-time adaptive interventions (JITAI) in mobile health: Key components and design principles for ongoing health behavior support. *Annals of Behavioral Medicine*, 52(6):446–462, 2018. DOI: <https://doi.org/10.1007/s12160-016-9830-8>.
- [10] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018. Available at: <http://incompleteideas.net/book/the-book-2nd.html>.
- [11] Ambuj Tewari and Susan A. Murphy. From ads to interventions: Contextual bandits in mobile health. In James M. Rehg, Susan A. Murphy, and Santosh Kumar, editors, *Mobile Health: Sensors, Analytic Methods, and Applications*, pages 495–517. Springer, 2017. DOI: https://doi.org/10.1007/978-3-319-51394-2_25.
- [12] Jiayu Yao, Emma Brunskill, Weiwei Pan, Susan Murphy, and Finale Doshi-Velez. Power constrained bandits. In *Proceedings of the 6th Machine Learning for Healthcare Conference (PMLR 149)*, pages 209–259, 2021. Available at: <https://proceedings.mlr.press/v149/yao21a.html>.
- [13] Elad Yom-Tov, Guy Feraru, Mark Kozdoba, Shie Mannor, Moshe Tennenholtz, and Irit Hochberg. Encouraging physical activity in patients with diabetes: Intervention using a reinforcement learning system. *Journal of Medical Internet Research*, 19(10):e338, 2017. DOI: <https://doi.org/10.2196/jmir.7994>.

A. Requirement coverage summary

The design maps every requirement to a concrete module or mechanism. Functional requirements [FR1–FR82](#) are owned by the feature modules of Table 5 (with the AI-planning cluster [FR5–FR25](#) realized by the engine of Section 4, and the lone cross-cutting navigation item [FR53](#) owned by the Core routing layer); non-functional requirements [NFR1–NFR39](#) are realized by the mechanisms of Table 3, the concurrency model, the resilience ladder, and the cloud-phase decisions of Table 13. The tables reproduced in this document are illustrative excerpts; the complete requirement-to-mechanism and requirement-to-test matrices (all 82 FR and 39 NFR) are maintained in the project repository, and no requirement is left unmapped. Table 26 is a consolidated view: the principal functional-requirement clusters, each mapped to the component that owns it (design) and the test-identifier area that verifies it (verification).

Table 26: Requirements traceability matrix (FR cluster → owning component → verification area).

Require-ments	Owning component	Verification area
FR1–FR4	onboarding	ONBOARD-*, PROFILE-*
FR5–FR15	today / daily_plan + AI engine	PLAN-*, FACTOR-*, TODAY-*
FR8, FR9, FR24 (cross-cut)	AI engine safety gate	SAFETY-*, BANDIT-ELIG-*
FR16–FR25	session + reward loop	SESSION-*, RECON-*, RPE-*
FR26–FR28	sessions_catalog	CATALOG-*
FR29–FR32	progress	PROGRESS-*, ACTIVE-DAYS-*
FR33–FR36	weather	WEATHER-CUBIT-*
FR37–FR39	shared/widgets (adaptive shell)	LAYOUT-*, NAV-*
FR48–FR52	settings	SETTINGS-*
FR53	Core routing	ROUTER-*
FR54–FR57, FR77	auth (cloud)	AUTH-*, BACKUP-*, DELETE-*
FR58–FR62	subscription (cloud)	IAP-*, ENTITLE-*
FR63–FR76	social (cloud)	SOCIAL-*, SHARED-*, pgTAP

B. Test identifier scheme

Automated tests use the identifier {AREA}-{TYPE}-{NNN}, where AREA is the feature or component under test (e.g. SAFETY, BANDIT, FACTOR, RECON), TYPE is the test level or concern (e.g. SVC, CUBIT, VIEW, UNIT, WIDGET), and NNN is a sequence number. This scheme makes the requirement-to-test traceability of Table 18 mechanical: a failing identifier points directly at the acceptance criterion (and therefore the requirement) it guards.

C. Reproducing the campaign

The headline figures of Table 22 are reproducible from a clean checkout, because the toolchain is pinned and the generated sources are hermetic. The build uses **Flutter 3.41.6** (stable channel) with **Dart 3.11.4**, under the package SDK constraint `sdk: ^3.11.3`. The Continuous Integration gate (GitHub Actions, `ubuntu-latest`) runs on every push and pull request to `main`; the same sequence reproduces the numbers locally from the `pulse_coach/` directory:

```
flutter pub get           # resolve deps; regenerate freezed / drift / gen_l10n sources
(cd wear && flutter pub get) # the nested Wear OS package resolves its own deps
flutter analyze           # static analysis -> 0 issues
flutter test              # 183 files, 1,421 cases -> 1,420 pass / 1 skip / 0 fail
```

The single skipped case is the iOS HealthKit path, which needs a real device. Generated files (`*.freezed.dart`, `*.g.dart`, and the drift / `gen_l10n` outputs) are not tracked and are regenerated by `flutter pub get`, so the checkout is self-contained. The cloud-phase database layer is verified separately against a local Supabase

instance:

```
supabase start && supabase test db # 18/18 pgTAP subtests pass
```

The offline policy-evaluation of Section 17.14 is a standalone, deterministic tool (it imports no Flutter and is excluded from the test count); it reproduces Table 24 from its seed:

```
dart run tool/policy_evaluation_sim.dart 42 # seed is optional (defaults to 42)
```

D. Explanation vocabulary

Because explainability is the product’s central trust mechanism (FR13–FR15), explanations are projected from stable localized *keys* rather than free text (Section 4). Table 27 reproduces a representative slice of the key catalogue (English rendering; Italian is the parallel default). Each behavioural state carries a short status label and a set of transition messages; environmental factors that were decisive in a given plan surface as factor labels. The full catalogue lives in the `app_en.arb` / `app_it.arb` resource bundles.

Table 27: Representative explanation keys and their English rendering.

Key	English rendering
<code>stateLabelActive</code>	“Ready”
<code>stateLabelFatigued</code>	“Under load”
<code>stateLabelAtRisk</code>	“At risk”
<code>stateLabelRecovering</code>	“Recovering”
<code>transitionActiveAtRisk</code>	“We missed you. Let’s restart light — 5 minutes is enough today.”
<code>transitionActiveFatigued</code>	“You pushed hard. Today we lighten up: short session.”
<code>transitionAtRiskRecovering</code>	“You are getting back into rhythm. Let’s keep it calm.”
<code>transitionRecoveringActive</code>	“You are back in shape! Let’s resume the full plan.”
<code>factorLabelTemperature</code>	“Temperature: {value}”
<code>factorLabelPrecipitation</code>	“Chance of rain”
<code>factorLabelAqi</code>	“Air quality not ideal”